HUJI · 67594 · INTRODUCTION TO COMMUNICATION NETWORKS

# Week 02

## Layering; Multiple Access on a Shared Link

modularity · protocols · the five layers · encapsulation · TDMA/FDMA · ALOHA · CSMA/CD · Ethernet · CSMA/CA · RTS/CTS · OSI · goodput · binomial vs. Poisson ALOHA

**What this week contains.** Part A is the lecture ("2 layers+broadcast", 103 slides), written as a textbook chapter. Part B is Recitation 2 ("ALOHA", 50 slides): the OSI model, goodput, slotted and pure ALOHA in the binomial and Poisson models, and two exam-style questions, all solved step by step. Practice problems and a cheat sheet follow. The lecture's "Probability 101" block (slides 49–67) repeats Tutorial 1, which Week 01 Part B solves in full. Here it gets a short recap, plus the one new mathematical tool that the MAC analysis needs, the limit $(1-1/N)^N \to 1/e$.

CONTENTS

PART A · LECTURE 2

1 Modularity: why networks are built in pieces 3
2 Layering 4
3 Protocols 6
4 Characterizing the layers 10
5 Building the Internet bottom up: the link layer 12
6 The multiple access problem 13
7 Channel partitioning: TDMA and FDMA 16
8 Random access protocols 17
9 Probability toolkit (recap of slides 49–67) 17
10 Slotted ALOHA 18
11 Pure (unslotted) ALOHA 20
12 CSMA: listen before you talk 22
13 CSMA/CD: collision detection 23
14 Summary of MAC protocols 26
15 Ethernet's CSMA/CD algorithm 27
16 MAC addresses 28
17 Ethernet 29
18 WiFi (IEEE 802.11): CSMA/CA 30
19 Summary: MAC protocols in practice 32

PART B · RECITATION 2

20 The OSI model: seven layers 34
21 Random access: the setting 35
22 Bandwidth, throughput and goodput 35
23 ALOHA: a brief history 37
24 The general model 37
25 Slotted ALOHA, binomial model 38
26 Pure ALOHA, binomial model 40
27 The Poisson models 41
28 Question 1: one host, two networks 42
29 Question 2: slotted and pure nodes on one channel 44

WEEK 02

30 Practice problems with full solutions 47
31 Cheat sheet 49

PART A · LECTURE 2

# Layering and the Broadcast Link

SECTION 01

# Modularity: why networks are built in pieces

A network is one of the largest systems ever built. Nobody could design it as a single program. Like every large computer system, it is split into *modules*.

**DEF · MODULE, INTERFACE, ABSTRACTION**

- A module is a part of a system that does one well-defined job.
- Its interface is the list of things other modules may ask it to do, and what they get back.
- An abstraction is what the interface lets you *forget*: you use the module without knowing how it works inside.

**EXAMPLE · MODULARITY YOU ALREADY USE**

- A **library**: you call `sort(list)` without knowing whether it runs quicksort or mergesort.
- A **programming language and its compiler**: you write `x = y + z` without knowing the CPU's registers and instruction set. The same program compiles for an Intel laptop and an ARM phone.

**WHAT WELL-DEFINED INTERFACES BUY YOU**

- **Hidden implementation**: a module's inside can be changed freely, and the change stays local.
- **Extensibility**: new functionality is added by adding new modules.
- **Evolution**: the system can keep improving piece by piece, for decades.
- **Isolated assumptions**: each module's assumptions stay inside it, and only high-level abstractions are shown outside.

**PITFALL · MODULARITY IS NOT FREE**

Hiding information can hurt performance. A module that cannot see inside its neighbours cannot use what they know. For example, a module that does not know the link is wireless may treat a loss as congestion. The trade-off is **ease of implementation vs. performance**, and networking chooses ease almost every time.

## > 1.1 What is special about *network* modularity

In a single program all modules run on one machine. In a network the implementation is **distributed across many machines**, hosts and routers. So the designers must decide three things:

1. **How to break the system into modules**: the answer is layering (this lecture).
2. **Where each module is implemented**: at the hosts, or inside the network?
3. **Where state is stored**: the principle of fate-sharing (later in the course).

The lecture addresses each choice when it becomes relevant; this week we meet the first two.

SECTION 02

# Layering

## >2.1 The tasks, bottom up

To move data from one application to another, the network has to solve these tasks, from the physical bottom to the top:

1. electrons (or light, or radio waves) on the wire;
2. bits on the wire;
3. packets on the wire;
4. deliver packets across a **local** network (a LAN), using **local addresses**;
5. deliver packets across **many** LANs, using **global addresses**;
6. make sure the packets get there (for example, handle loss);
7. do something useful with the data.

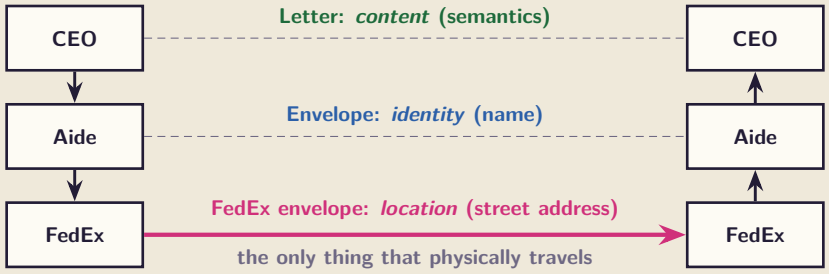
DEF · LAYERING

Layering organizes the tasks into a stack of layers. It is a particularly **strict form of modularity**: each layer interacts only with the layer directly above it and the layer directly below it. A layer uses the service of the layer below and offers a service to the layer above.

## >2.2 Layering in the real world: the CEO's letter

EXAMPLE · A LETTER BETWEEN TWO COMPANIES

1. CEO A writes a letter to CEO B, folds it and hands it to an administrative aide.
2. The aide puts the letter in an envelope with CEO B's name on it and takes it to FedEx.
3. The FedEx office puts the envelope in a larger FedEx envelope, writes the company's name and street address on it, and puts it on a truck.
4. FedEx delivers it to company B. There, the FedEx envelope is opened, the aide reads the name and passes the inner envelope to CEO B, and CEO B reads the letter.



The path of the letter (slide 10). Each dashed line joins two *peers* that understand the same thing. The real movement goes down on the left, across at the bottom, and up on the right.

THREE LESSONS OF THE LETTER

- **Peers understand the same things, and no one else does.** The FedEx driver never reads the letter; the CEO never reads the street address.
- **Each layer adds its own wrapping**, so the lowest layer carries the most packaging.
- **Each layer only talks to its neighbours**: the CEO deals with the aide, never with the truck.

## > 2.3 The resulting five layers

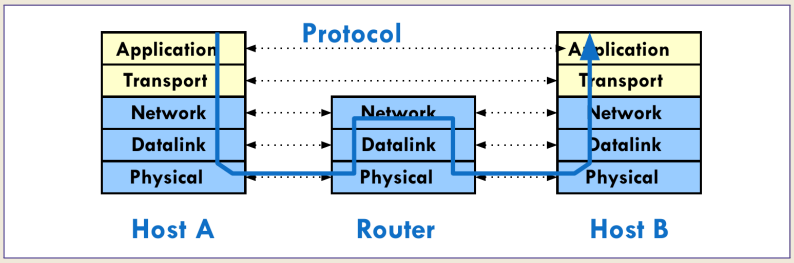
Each task becomes a layer. Two tasks are folded into the layer above them.

| LAYER | TASK | WHAT IT IS ABOUT |
|---|---|---|
| 5 Application | do something with the data | web, e-mail, video, games |
| 4 Transport | ensure the packets get there | between *processes*; reliability |
| 3 Network | deliver across many LANs | **global** addresses; routing |
| 2 (Data) Link | deliver across one LAN (incl. packets on wire) | **local** addresses; sharing the medium |
| 1 Physical | bits on the wire (incl. electrons) | voltages, light, radio |

A MNEMONIC FOR THE FIVE

Read from the bottom: **P**hysical, **L**ink, **N**etwork, **T**ransport, **A**pplication. Each layer answers one question: *how is a bit sent? to which neighbour? to which machine in the world? to which program on it? and what does the program do with it?*

## > 2.4 Who implements which layers?




Slides 13–15. Host A and Host B implement all five layers; the router in the middle implements only the bottom three. The blue line is the *physical* path of the data: down Host A's stack, across the wire, up to the router's network layer, down again, and up Host B's stack.

THE TOP TWO LAYERS LIVE ONLY AT THE HOSTS

Routers implement **physical, link and network**. **Transport and application are implemented only at the end hosts**. Inside the network, nobody looks at the transport or application data. (This is the answer to the second design question of §1.1: *where* each module is implemented.)

DEF · LOGICAL VS. PHYSICAL COMMUNICATION

- Logical communication: each layer behaves as if it talks directly to its peer, the same layer on the other machine. Host A's transport layer "talks to" Host B's transport layer. The router's network layer "talks to" both hosts' network layers.
- Physical communication: the bits really go *down* to the physical layer, travel peer to peer across each link, and go back *up* to the relevant layer.

SECTION 03

# Protocols

DEF · PROTOCOL

A protocol is an **agreement on how to communicate**: to exchange data, and to coordinate the sharing of resources. A protocol specifies:

- its syntax: how the protocol is structured, meaning the format of the messages and the order in which they are sent and received;
- its semantics: what the bits mean, meaning how to respond to each message or event.

EXAMPLE · PROTOCOLS IN LIFE

- **Asking a question in class**: raise a hand, wait to be called on, then speak.
- **Turn-taking in conversation**: a pause is the signal that the next person may respond.
- **Boarding and leaving an airplane**: by rows or zones. Not all countries use the same protocol!
- **Answering the phone**: you say "hello" as the signal for the caller to talk; the caller identifies themselves, and then the conversation starts.

Each one has syntax (what is said, in what order) and semantics (what each signal means and what to do next).

EXAMPLE · HTTP, AN APPLICATION-LAYER PROTOCOL

A browser asks a web server for a page. The request (syntax: a method, a path, a version, then header lines, then an empty line):

```
GET /courses/archive/spring06/cos461/ HTTP/1.1
Host:  www.cs.princeton.edu
User-Agent:  Mozilla/4.03
(empty line)
```

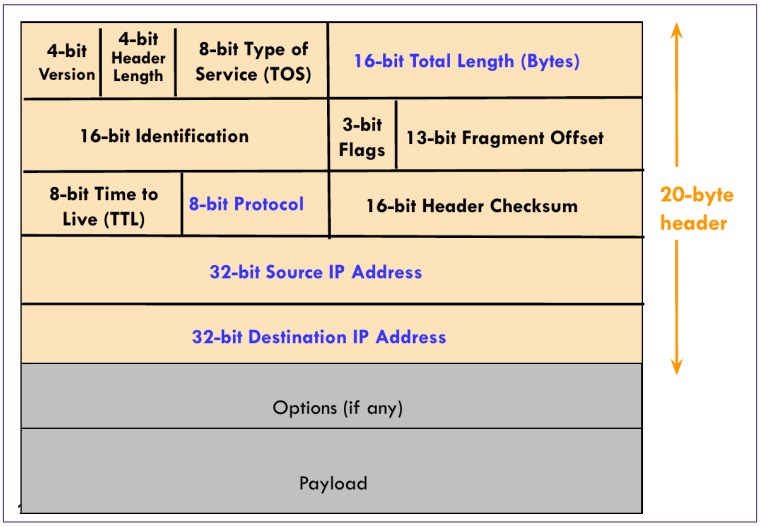
The response (semantics: `200 OK` means "here it is"):

```
HTTP/1.1 200 OK
Date:  Mon, 6 Feb 2006 13:09:03 GMT
Server:  Netscape-Enterprise/3.5.1
Last-Modified:  Mon, 6 Feb 2006 11:12:23 GMT
Content-Length:  21
(empty line)
Site under construction
```

`Content-Length:  21` tells the browser exactly how many bytes of body to read: "Site under construction" is 23 characters, so the slide's example is slightly off, but the idea is that the header announces the length.

EXAMPLE · IP, A NETWORK-LAYER PROTOCOL

Lower layers use binary formats with fixed-size fields. The IP packet header:




Slide 20. Each row is 32 bits. Without options the header is 5 rows = 20 bytes. The fields you need now: *Version* (4 for IPv4), *Total Length*, *Time to Live* (decremented by each router; the packet is dropped at 0), *Protocol* (which transport protocol is inside, e.g. TCP), and the 32-bit *source* and *destination* addresses.

## > 3.1 Standardization

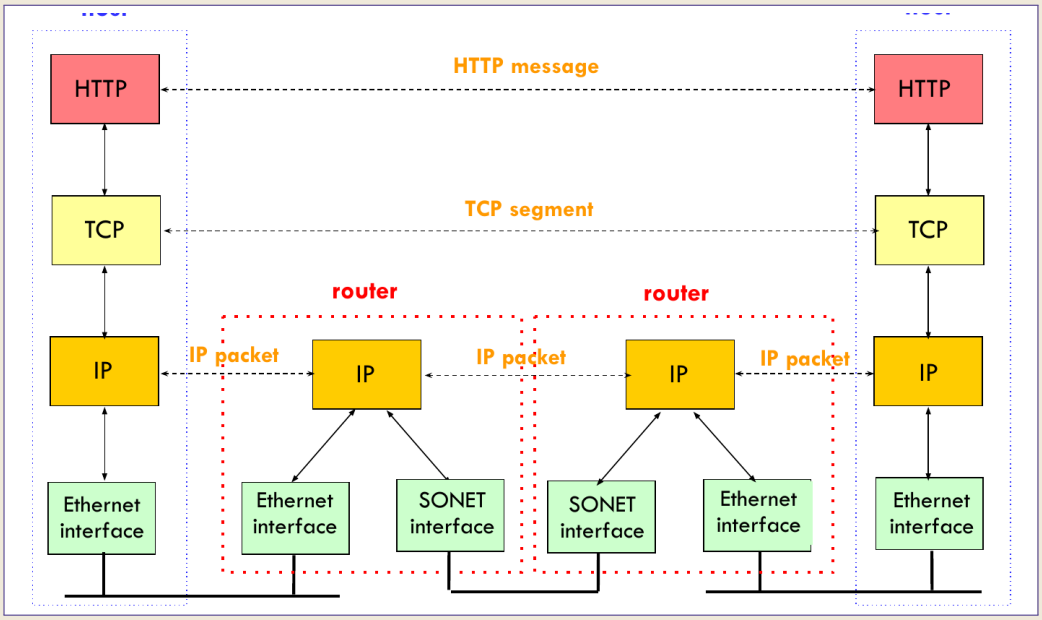
WHY STANDARDS

Everyone must follow the same protocol. Very small changes can make a big difference, or stop communication altogether: if one side thinks the length field is 16 bits and the other 32, nothing works. Hence **standards**. A standard fixes the protocol, *not* the code, so there can be many independent implementations (Windows, Linux and macOS each have their own TCP).

DEF · IETF AND RFCS

The Internet Engineering Task Force (IETF, `ietf.org`) writes the Internet's standards. It works in *working groups* focused on specific issues, and publishes its documents as numbered Requests For Comments (RFCs), archived at `rfc-editor.org`. (Link-layer standards such as Ethernet and WiFi come from the IEEE 802 committees.)

## > 3.2 The Internet protocol suite, end to end

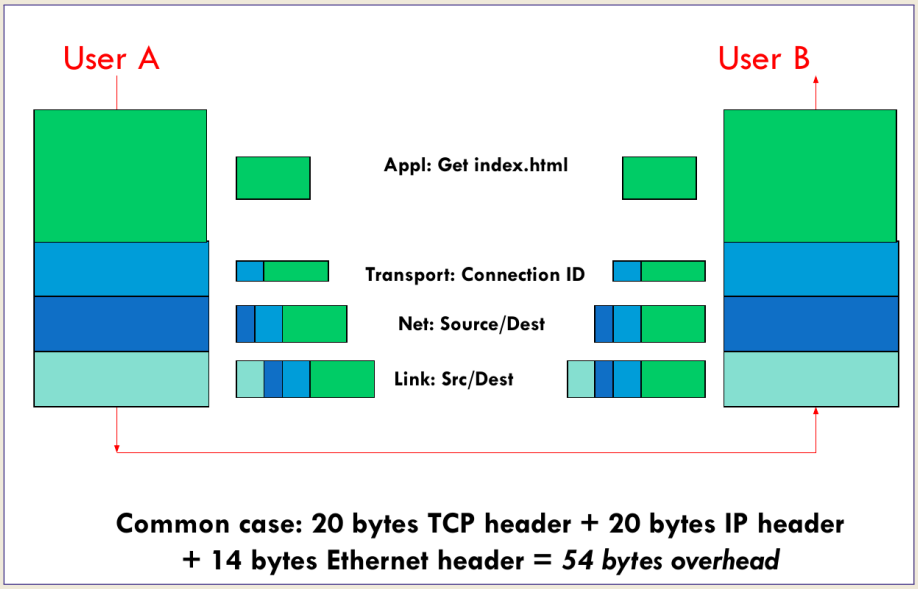



Slide 22. Each layer's unit of data has its own name: the application sends an *HTTP message*, the transport layer a *TCP segment*, the network layer an *IP packet* (datagram), and the link layer a *frame*. The two routers process only IP and below. Notice that the link changes along the way: Ethernet, then SONET, then Ethernet again. IP runs over all of them.

## > 3.3 Encapsulation

DEF · ENCAPSULATION

Going down the stack, each layer takes what it received from above as its *payload* and adds its own *header* in front. This is encapsulation. Going up the stack at the receiver, each layer removes its own header and passes the payload up (decapsulation).




Slide 23. User A's request "Get index.html" (green) gets a transport header (connection ID), then a network header (source and destination addresses), then a link header (local source and destination). At User B the headers are removed in the opposite order.

**Step by step for one web request** [KR 1.5.2]:

1. The application builds the HTTP message $M$.
2. Transport adds a header $H_t$ (which process on which host): segment $= H_t\, M$.
3. Network adds $H_n$ (global source and destination addresses): datagram $= H_n\, H_t\, M$.
4. Link adds $H_l$ (local source and destination on this LAN): frame $= H_l\, H_n\, H_t\, M$.
5. The physical layer sends the bits. Each router strips $H_l$, reads $H_n$, and wraps the datagram in a *new* link header for the next link.

**EXAMPLE · HOW MUCH IS OVERHEAD?**

Common case: 20 bytes TCP + 20 bytes IP + 14 bytes Ethernet = 54 bytes of headers.

- A full frame with 1460 bytes of data: $1460/(1460+54) = 96.4\%$ of the bytes are useful.
- A single typed keystroke (1 byte of data, e.g. in a remote shell): $1/55 = 1.8\%$ useful. Mice pay heavily for headers.

**PITFALL · 54 BYTES COUNTS HEADERS ONLY**

The 54 bytes are the three *headers*. On the wire, Ethernet also adds a 4-byte CRC *trailer* after the data (and an 8-byte preamble before the frame) [KR 6.4.2]. So the real per-frame overhead is a bit larger.

SECTION 04

# Characterizing the layers

DEF · SERVICE, INTERFACE, PROTOCOL (OSI TERMS)

Every layer is described by three things:

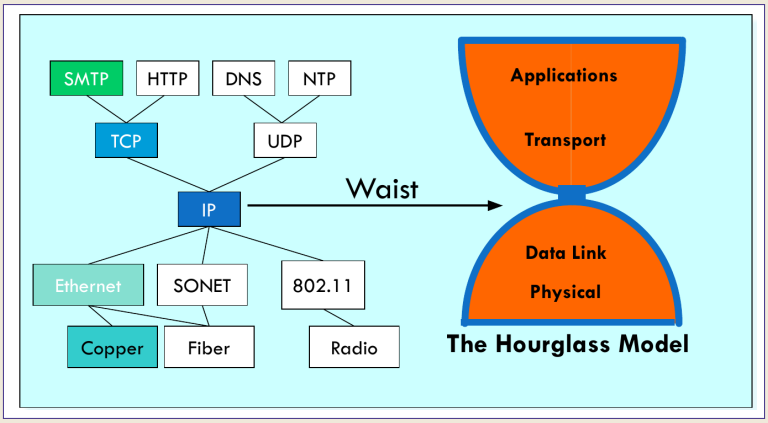
- its service: *what* the layer does for the layer above;
- its service interface: *how* the layer above accesses the service;
- its protocol: how the *peers* on different machines communicate to provide the service. The protocol governs what happens *between* machines, not how the layer is coded on one machine.

A layer can have **many implementations**, and this is what allows innovation.

| LAYER | SERVICE | PROTOCOLS DO | EXAMPLES |
|---|---|---|---|
| 1 Physical | move bits between two systems on *one* physical link | coding of bits: voltage levels, bit duration | optical, wireless (not our focus) |
| 2 Link | exchange frames with hosts on the same LAN, using local addresses | Media Access Control (MAC) | Ethernet, 802.11, Frame Relay, ATM |
| 3 Network | deliver packets to any destination, across many layer-2 networks, using global addresses | routing | IP (only!) |
| 4 Transport | communication between *processes*; perhaps reliability, rate adaptation | reliability, flow control, splitting large messages | TCP, UDP |
| 5 Application | any service to the end user | depends on the application | HTTP, SMTP, Skype, BitTorrent |

- **Link interface**: "send this frame to that local host"; "deliver to me the frames addressed to me". The link layer is not limited to a single wire: a LAN may have many hosts.
- **Network interface**: "send this packet to that global destination"; "deliver to me the packets for this host". "Inter-network" means across *different* layer-2 networks, e.g. from Ethernet to 802.11.
- **Transport interface**: "send this message to that process on that host"; a local process receives the messages sent to it. Multiplexing here means that many processes on one host share the host's network connection.

## > 4.1 The narrow waist




The Internet hourglass (slide 30). Many applications at the top, many link and physical technologies at the bottom, and exactly one protocol in the middle: IP.

**WHY ONLY ONE NETWORK-LAYER PROTOCOL**

The slide asks: "IP: why no other examples?" Because **IP is the narrow waist**. Every application runs over IP, and IP runs over every link technology.

- A new application (top) only has to work with IP, and it runs on every network.
- A new link technology (bottom) only has to carry IP, and every application works over it.

With $A$ applications and $L$ link technologies, a single waist needs $A + L$ adaptations instead of $A \times L$. With 100 applications and 20 links: 120 instead of 2000. The narrow waist is what makes interoperability possible. The price: changing IP itself is extremely hard (IPv6 has been "coming" for decades).

**PITFALL · "WASTE" VS. "WAIST"**

Slide 30's title says "The Narrow *Waste* of the Internet". The word is **waist**, as in the narrow middle of an hourglass, which is what the figure shows.

SECTION 05

# Building the Internet bottom up: the link layer

The course now builds the Internet from the bottom. **Challenge I: communicating within a LAN.** For now we look only at the physical and link layers (the physical layer itself is not the focus of the course). Transport and network come later; the link layer is needed to interconnect LANs.

## > 5.1 Terminology

DEF · NODE, LINK, FRAME

- Nodes: end hosts *and* relays (routers, switches): anything that runs a link layer.
- Links: the communication channels that connect *adjacent* nodes along the path. They can be wired, wireless, or logical (a broadcast domain).
- Frame: the layer-2 packet. It *encapsulates* a network-layer datagram.

The **data-link layer** is responsible for transferring a datagram from one node to an *adjacent* node over one link.

EXAMPLE · THE TRIP FROM HONOLULU TO JERUSALEM [KR 6.1]

A taxi from Honolulu to HNL airport, a plane to Newark (EWR), a plane to Tel Aviv (TLV), a train to Jerusalem.

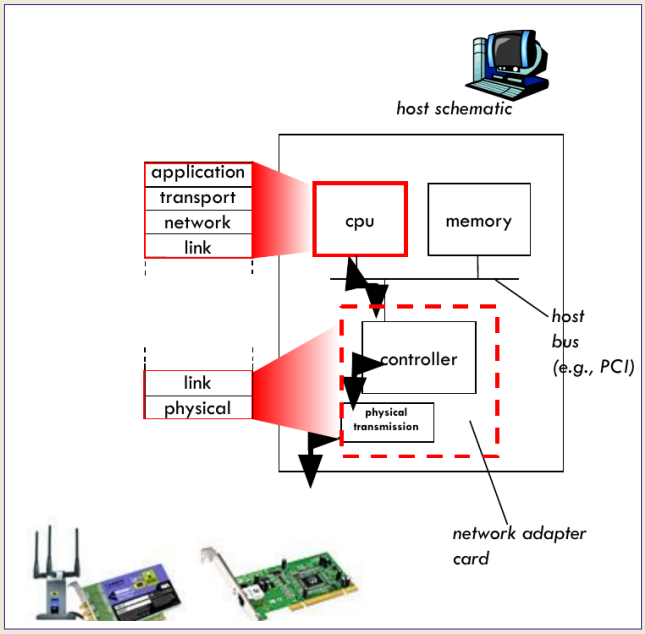
| TRAVEL | NETWORK |
|---|---|
| passenger | datagram |
| each segment of the trip | one link |
| transportation mode (taxi, plane, train) | link-layer protocol |
| travel agent who plans the route | routing algorithm |

The datagram crosses different links with *different* link protocols (Ethernet on the first link, 802.11 on the next), and each one provides different services. A taxi does not promise what a plane does, and some links are reliable while others are not.

## > 5.2 Services of a single link

- **Link access and addressing.** When the channel is a shared medium, someone must decide who sends. MAC addresses in the frame header identify the source and the destination. They are different from IP addresses (§16).
- **Reliable delivery between adjacent nodes** (error detection and correction, retransmissions). It is seldom used on low-bit-error links (fibre, some twisted pair), but needed on wireless links, which have high error rates. This is revisited later in the course.
- "...?": the slide leaves the list open. [KR 6.1.1] also lists *framing* (wrapping the datagram) and *error detection* as separate services.

### >5.3 Where is the link layer implemented?




Slide 39. Inside a host, application, transport and network run in software on the CPU. Link and physical are in the *network adapter*.

**DEF · NETWORK INTERFACE CARD (NIC)**

The link layer runs **in every host**, in an adapter, also called a network interface card (NIC): e.g. an Ethernet card or an 802.11 card. The NIC implements the link and physical layers. It attaches to the host's system bus and is a combination of hardware, software and firmware.

**ADAPTERS COMMUNICATING**

- **Sending side**: encapsulates the datagram in a frame and adds error-checking bits (and, if used, reliable-delivery machinery).
- **Receiving side**: checks for errors, extracts the datagram and passes it up to the network layer.

SECTION 06

# The multiple access problem

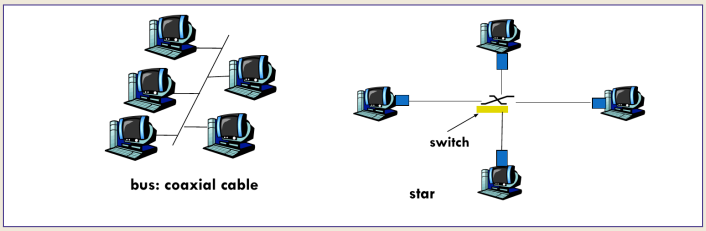
**DEF · BROADCAST CHANNEL AND COLLISION**

On a single shared broadcast channel, every frame sent by one node is received by all the others. If a node receives two or more signals at the same time, there is a collision: the signals are tangled together and all the colliding frames are lost, and the channel was wasted during the collision [KR 6.3].

DEF · MULTIPLE ACCESS PROTOCOL

A multiple access protocol (MAC protocol) is a **distributed algorithm** that determines how the nodes share the channel, i.e. *when* each node may transmit. The catch: the communication about how to share the channel must use the channel itself. There is no separate "out-of-band" channel for coordination.

## > 6.1 Broadcast domains: wired



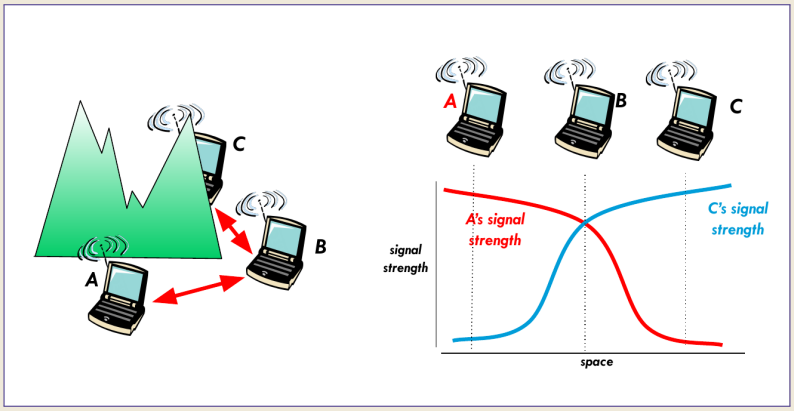

Slides 42, 96. Left: a *bus*, one coaxial cable shared by every node. Right: a *star*, every node on its own cable to a switch.

- **Bus** (popular until the mid-1990s): all nodes are in the same collision domain: any two of them can collide.
- **Star** (today): an *active switch* sits in the centre. Each node runs a separate Ethernet with the switch, so nodes do not collide with each other. Switches come later in the course.

PITFALL · COLLISION DOMAIN VS. BROADCAST DOMAIN

Slide 42 says the bus nodes are in the same "collision domain"; slide 96, the same slide repeated, says "broadcast domain". On a bus the two coincide, so both are true. In general they are different things: a *collision domain* is the set of nodes whose transmissions can collide; a *broadcast domain* is the set of nodes that receive a broadcast frame. The switched star still has one broadcast domain, but each cable is its own collision domain.

## > 6.2 Broadcast domains: wireless




Slide 43. Left: A and C are separated by a mountain; both reach B, but they cannot hear each other. Right: A's signal (red) fades with distance; so does C's (blue). At B both are strong enough; at A, C's signal is too weak to sense, and vice versa.

**DEF · HIDDEN TERMINAL AND FADING** [KR 7.2]

In wireless, "everyone hears everyone" is not true.

- Hidden terminal problem: A and C both reach B, but an obstacle hides them from each other. Neither can tell that the other is transmitting, so they collide at B.
- Fading: signal strength decreases with distance. Even without an obstacle, A and C may be too far apart to hear each other, although B, in the middle, hears both.

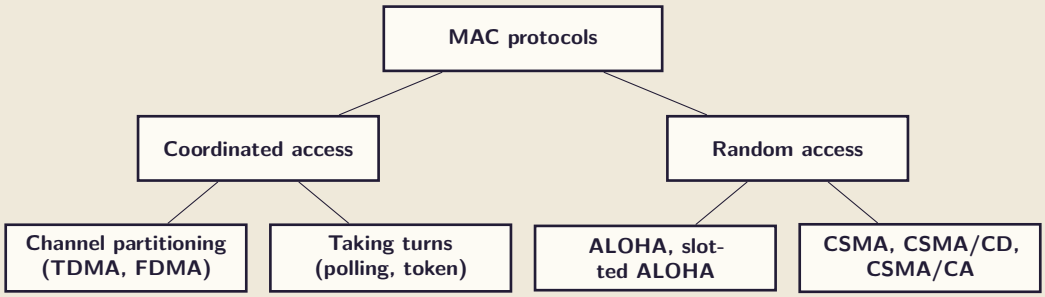
## > 6.3 The ideal multiple access protocol

**WISH LIST FOR A BROADCAST CHANNEL OF RATE $R$ BIT/S**

1. When **one** node wants to transmit, it can send at rate $R$.
2. When $M$ nodes want to transmit, each can send at **average** rate $R/M$.
3. **Fully decentralized**: no special node coordinates (no single point of failure).
4. **No synchronization** of clocks or slots.
5. **Simple**.

Keep this list in mind: every protocol below is scored against it in §14.1.

## > 6.4 A taxonomy of MAC protocols




The lecture's two broad classes (slides 45, 89, 103).

- Channel partitioning: divide the channel into smaller "pieces" (time slots or frequencies) and give each node a piece for its exclusive use.
- Taking turns: nodes take turns, but nodes with more to send can take longer turns.
- Random access: the channel is not divided. Collisions are allowed, and the protocol *recovers* from them.

**PITFALL · IS CHANNEL PARTITIONING "CENTRALIZED"?**

The slides group channel partitioning under "Centralized Coordination". Once the slots or bands are assigned, TDMA and FDMA need *no* coordinator at run time: every node simply uses its own piece. What they need is an *agreement made in advance*. Only some taking-turns protocols (polling) have a real central master. That is why [KR 6.3] lists **three** classes side by side: channel partitioning, random access and taking turns. Also, slide 103 writes

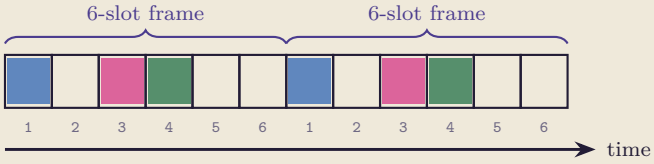
"time/frequency *modulation*". It should be time/frequency **division** (multiplexing). Modulation is how bits are put on a signal, a physical-layer matter.

SECTION 07

# Channel partitioning: TDMA and FDMA

**DEF · TDMA: TIME DIVISION MULTIPLE ACCESS**

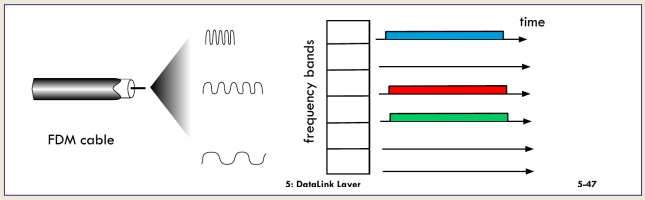
Access to the channel goes in **rounds** (frames). Each station gets a fixed-length slot in every round; the slot length is one packet transmission time. Unused slots go idle.




Slide 46's example: a 6-station LAN in which stations 1, 3 and 4 have packets. Slots 2, 5 and 6 stay idle in every frame, even though 1, 3 and 4 have more to send.

**DEF · FDMA: FREQUENCY DIVISION MULTIPLE ACCESS**

The channel's spectrum is divided into frequency bands, and each station is assigned a fixed band. Unused transmission time in a band goes idle.




Slide 47. One FDM cable carries six bands. Stations 1, 3, 4 send in their bands; bands 2, 5, 6 are idle.

**EXAMPLE · THE COST OF PARTITIONING, WITH NUMBERS** [KR 6.3.1]

A channel of $R = 60$ Mbps shared by $N = 6$ stations: each one gets exactly $R/N = 10$ Mbps.

- 6 stations busy: each gets 10 Mbps $= R/M$. Property 2 is met perfectly, and there are no collisions.
- Only station 1 busy: it still gets only 10 Mbps, and 50 Mbps are wasted. Property 1 fails.
- Even when alone, a TDMA station must wait for its own slot: on average half a frame of delay before each packet.

Like a pan of Kaiserschmarrn torn into six fixed portions before the guests arrive: if only one guest is hungry, they still get one sixth, and the rest goes cold.

SECTION 08

# Random access protocols

**DEF · RANDOM ACCESS**

When a node has a packet to send, it transmits at the **full channel rate** $R$, with no prior coordination among the nodes. If two or more nodes transmit, there is a collision. A random access MAC protocol specifies:

- how to **detect** collisions;
- how to **recover** from them, e.g. by delayed retransmission.

Examples: slotted ALOHA, (pure) ALOHA, CSMA, CSMA/CD, CSMA/CA.

**WHY THE DELAY MUST BE RANDOM**

If two colliding nodes both waited the *same* fixed time and tried again, they would collide again, forever. Each node chooses its delay *independently at random*, so with luck one of them goes first and gets through [KR 6.3.2].

SECTION 09

# Probability toolkit (recap of slides 49–67)

The lecture now pauses for "Probability 101": sample spaces, events, probability functions, conditional probability, Bayes, random variables, expectation, the four discrete distributions, and Examples I–III. **Week 01 Part B covers all of these in full**, including Example II (good and bad packets) and Example III (the chain of links). Here are only the facts that the MAC analysis below uses.

**WHAT WE NEED**

- **Independent events multiply**: $P(A \cap B) = P(A)P(B)$. So "$N-1$ independent nodes all stay silent, each with probability $1-p$" has probability $(1-p)^{N-1}$.
- **Disjoint events add**: $P(A \cup B) = P(A) + P(B)$.
- **Exactly one success among** $N$: $\binom{N}{1}p(1-p)^{N-1} = Np(1-p)^{N-1}$, the binomial probability $P(X=1)$ for $X \sim \mathrm{Bin}(N,p)$.
- **Geometric**: if each try succeeds with probability $q$, the number of tries is $\mathrm{Geo}(q)$, with mean $1/q$. The expected number of *failures* before the success is $1/q - 1 = (1-q)/q$.

**EXAMPLE · EXAMPLE I FROM THE SLIDES: 50 COMPONENTS**

Communication uses 50 components, each working 99% of the time (independently). Failure probability $= 1 - 0.99^{50} \approx 39.5\%$. With 99.9%: $1 - 0.999^{50} \approx 4.9\%$. The system must be

designed to expect failure.

PITFALL · SLIDE 67: "THE PROBABILITY TO SUCCESS DROPS TO $(1-p)$"

In Example III (Week 01, Question 3) the success probability goes from $(1-p)^n$ to $(1-p)$. Since $1-p > (1-p)^n$ for $n \geq 2$, it **rises**, not "drops". Also, the slide's $\mathbb{E}X = 1/(1-p)$ counts the sends needed to cross *one* hop. Crossing all $n$ hops hop by hop takes $n/(1-p)$ link transmissions on average.

### >9.1 The new tool: $(1-1/N)^N \to 1/e$

The efficiency of ALOHA ends with "take the limit as $N \to \infty$". This is the limit it needs.

A FAMOUS LIMIT

$$\lim_{N\to\infty}\left(1-\frac{1}{N}\right)^N = \frac{1}{e} \approx 0.368, \qquad \text{and therefore also} \qquad \lim_{N\to\infty}\left(1-\frac{1}{N}\right)^{N-1} = \frac{1}{e}.$$

(The second follows from the first: $(1-\frac{1}{N})^{N-1} = (1-\frac{1}{N})^N/(1-\frac{1}{N})$, and the denominator tends to 1.)

EXAMPLE · SEE IT CONVERGE

| $N$ | $(1-1/N)^N$ | $(1-1/N)^{N-1}$ |
|---|---|---|
| 2 | 0.250 | 0.500 |
| 5 | 0.328 | 0.410 |
| 10 | 0.349 | 0.387 |
| 100 | 0.366 | 0.370 |
| $\infty$ | 0.368 | 0.368 |

Why it is true, in words: $\ln(1-x) \approx -x$ for small $x$, so $N\ln(1-\frac{1}{N}) \approx N\cdot(-\frac{1}{N}) = -1$, and $(1-\frac{1}{N})^N \approx e^{-1}$. In probability terms: if each of $N$ nodes stays silent with probability $1-\frac{1}{N}$, then the chance that *all* stay silent tends to $1/e$. It is the Poisson probability $P(X=0)$ for mean 1.

SECTION 10

# Slotted ALOHA

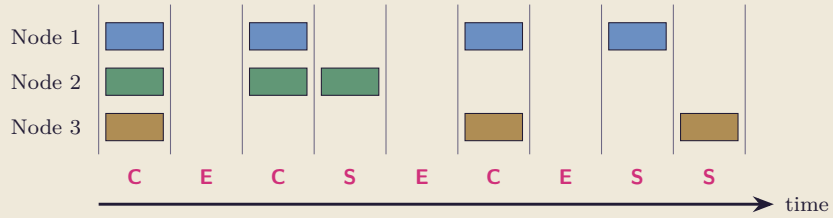
DEF · SLOTTED ALOHA

**Assumptions**: all frames have the same size; time is divided into equal slots, each the time to transmit one frame; nodes start transmitting only at the beginning of a slot; nodes are synchronized; if two or more nodes transmit in a slot, all nodes detect the collision.

**Operation**:

1. When a node gets a fresh frame, it transmits in the next slot.
2. No collision: success; the node may send a new frame in the next slot.
3. Collision: the node retransmits the frame in *each subsequent slot with probability* $p$, until it succeeds.

"With probability $p$": before each slot the node tosses a biased coin, and transmits on heads (probability $p$).




Slotted ALOHA in action [KR Fig. 6.10]. C = collision, E = empty, S = success. All three nodes collide in slot 1. Node 2 succeeds in slot 4, node 1 in slot 8 and node 3 in slot 9.

**Pros**: a single active node can transmit continuously at the full rate $R$; highly decentralized (only the slot times must be in sync); simple.
**Cons**: collisions waste slots; idle slots are wasted too; a node may detect a collision long before the slot ends but still wastes the whole slot; it needs clock synchronization.

## > 10.1 Goodput vs. throughput

DEF · GOODPUT AND THROUGHPUT

Consider many nodes, all with many frames to send.

- Goodput (also efficiency): the long-run fraction of *successful* slots.
- Throughput: the long-run fraction of slots in which there was a transmission, successful or not.

Throughput counts collisions as "busy"; goodput counts only useful work. A channel full of collisions has high throughput and zero goodput.

## > 10.2 The goodput of slotted ALOHA, step by step

**Simplified model.** $N$ nodes, each always has a frame, and in each slot *each* node transmits with probability $p$, independently (this treats fresh frames and retransmissions alike).

1. **A given node succeeds** in a slot iff it transmits *and* the other $N-1$ do not. By independence:

$$P(\text{node } i \text{ succeeds}) = p\,(1-p)^{N-1}.$$

2. **Some node succeeds**: the events "node $i$ succeeds" for $i = 1, \ldots, N$ are *disjoint* (two nodes cannot both succeed in one slot), so we add:

$$\alpha(p) = N\,p\,(1-p)^{N-1}.$$

3. **Best** $p$: differentiate and set to zero (slide 84):

$$\frac{d\alpha}{dp} = N(1-p)^{N-1} - pN(N-1)(1-p)^{N-2} = N(1-p)^{N-2}\left[(1-p)-(N-1)p\right] = 0 \;\Rightarrow\; p^* = \frac{1}{N}.$$

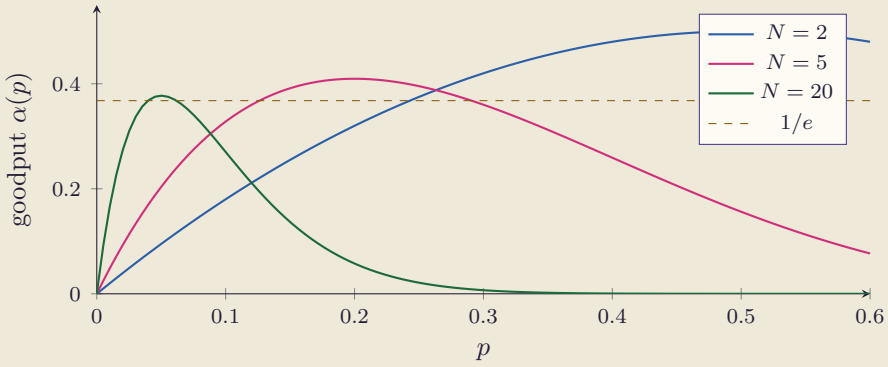
   Makes sense: on average exactly one node transmits per slot.

4. **Best goodput**: $\alpha(p^*) = N \cdot \frac{1}{N}\left(1-\frac{1}{N}\right)^{N-1} = \left(1-\frac{1}{N}\right)^{N-1}$.

5. **Many nodes**: by the limit above, $\alpha_{\max} \to \dfrac{1}{e} \approx 0.37$.

**PITFALL · SLIDE 73: "APPROX." $Np(1-p)^{N-1}$**

The slide calls $Np(1-p)^{N-1}$ "approximately" the probability that some node succeeds. Within the model it is **exact**: the $N$ success events are disjoint, so their probabilities add with no approximation. The approximation comes only in the last step, when $(1-\frac{1}{N})^{N-1}$ is replaced by its limit $1/e$.




Slotted ALOHA goodput as a function of $p$. Each curve peaks at $p = 1/N$; the peak heights 0.50, 0.41, 0.38 approach $1/e$.

**EXAMPLE · WHERE THE SLOTS GO [KR 6.3.2]**

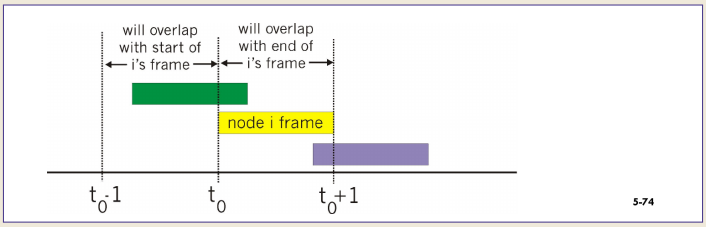
With many nodes at $p = 1/N$: 37% of slots succeed, 37% are empty ($(1-\frac{1}{N})^N \to 1/e$), and the remaining 26% are collisions. A 100 Mbps slotted ALOHA LAN with many busy users delivers less than 37 Mbps of useful data.

SECTION 11

# Pure (unslotted) ALOHA

**DEF · PURE ALOHA**

No slots and no synchronization: simpler. Each host behaves *as if* it ran slotted ALOHA on its *own* slots, sending a frame with probability $p$ in each of them. But the other hosts' "slots" are not aligned with its own.




Slide 74. Node $i$'s frame (yellow) occupies $[t_0, t_0 + 1]$ (the frame time is the unit of time). A frame started in $[t_0 - 1, t_0]$ (green) overlaps its start; a frame started in $[t_0, t_0 + 1]$ (purple) overlaps its end.

**Step by step.**

1. The vulnerable period is **two** frame times, $[t_0 - 1, t_0 + 1]$, instead of one.
2. $P(\text{node } i \text{ succeeds}) = P(\text{it transmits}) \cdot P(\text{no other in } [t_0 - 1, t_0]) \cdot P(\text{no other in } [t_0, t_0 + 1]) = p\,(1-p)^{N-1}(1-p)^{N-1} = p\,(1-p)^{2(N-1)}$.
3. Goodput $Np(1-p)^{2(N-1)}$. The best $p$ is $p^* = 1/(2N-1)$, and as $N \to \infty$ the maximum tends to $\dfrac{1}{2e} \approx 0.18$.

**HALF OF SLOTTED ALOHA**

Pure ALOHA's maximum is exactly half of slotted ALOHA's: 0.18 vs. 0.37. The vulnerable window is twice as long. That is the price of not synchronizing clocks.

| $N$ | SLOTTED, $(1-\frac{1}{N})^{N-1}$ | PURE, at $p^* = \frac{1}{2N-1}$ |
|---|---|---|
| 2 | 0.500 | 0.296 |
| 5 | 0.410 | 0.216 |
| 20 | 0.377 | 0.191 |
| $\infty$ | $1/e = 0.368$ | $1/(2e) = 0.184$ |

SECTION 12

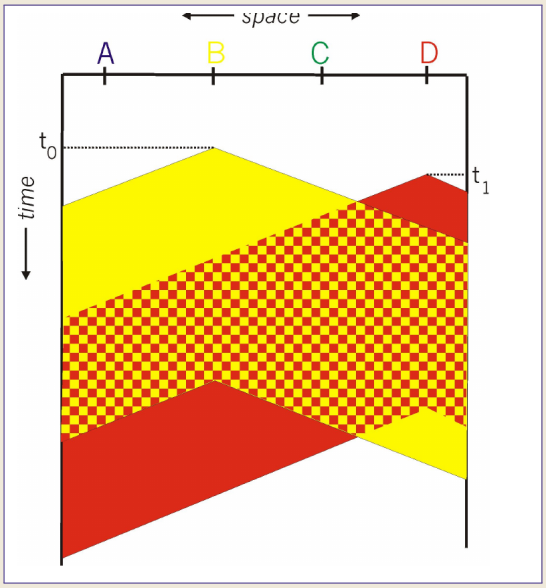
# CSMA: listen before you talk

**DEF · CSMA (CARRIER SENSE MULTIPLE ACCESS)**

- If the channel is sensed **idle**: transmit the entire frame.
- If the channel is sensed **busy**: defer the transmission.

Human analogy: *don't interrupt others!* ALOHA, in contrast, is the guest who keeps talking no matter who else is speaking.

## > 12.1 Why CSMA still has collisions




Slide 77, a space-time diagram: space horizontally (four nodes on a bus), time downwards. B starts at $t_0$; its signal (yellow) spreads in both directions. At $t_1$ B's signal has not yet reached D, so D senses an idle channel and starts too (red). The chequered area is where they collide.

**THE ROLE OF PROPAGATION DELAY**

Carrier sensing only tells a node what was on the channel *one propagation delay ago*. Two nodes may each sense "idle" while the other's signal is still on its way. The longer the propagation delay between nodes, the larger the collision probability. In plain CSMA, when a collision happens, **the whole frame transmission time is wasted**: B and D both keep transmitting to the end.

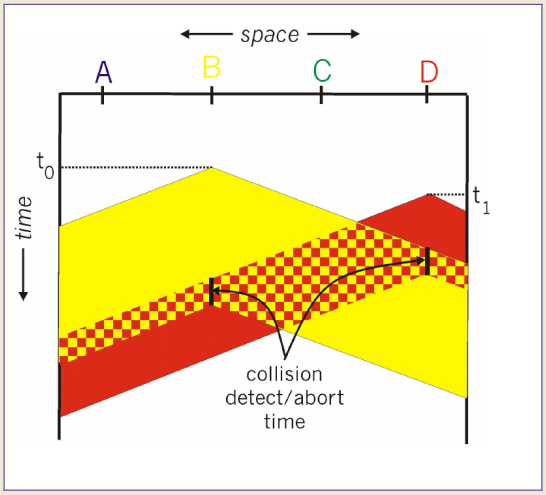
SECTION 13

# CSMA/CD: collision detection

DEF · CSMA/CD

Carrier sensing as in CSMA, plus collision detection: a node keeps listening while it transmits. Collisions are detected within a short time, and the colliding transmissions are **aborted**, which reduces the waste. Human analogy: the polite conversationalist who stops when someone else starts talking.

- **Easy in wired LANs**: compare the signal transmitted with the signal received.
- **Hard in wireless LANs**: the node's own transmission overwhelms the weak signal it receives from others.



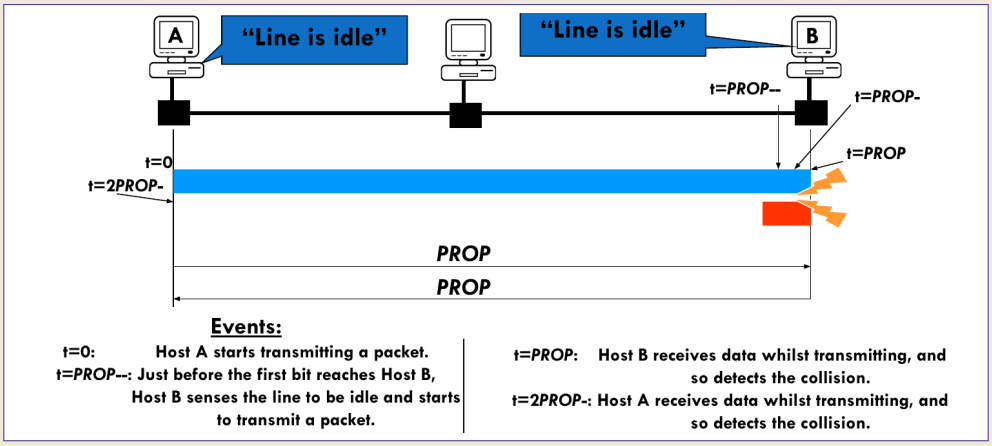

Slide 79: the same scenario with collision detection. B and D each stop shortly after the other's signal arrives. The wasted area is much smaller than in plain CSMA.

THE CSMA/CD PROTOCOL (SLIDE 80)

All hosts transmit and receive on one channel. When a host has a packet to send:

1. **Carrier sense**: check that the line is quiet before transmitting.
2. **Collision detection**: detect a collision as soon as possible. If one is detected, stop transmitting, wait a random time, and return to step 1.

## > 13.1 The worst case: how long until a collision is detected?




Slide 81. A and B are at the two ends of the cable, one propagation delay PROP apart.

**The events, step by step** ($\varepsilon$ is a tiny time):

1. $t = 0$: A senses idle and starts transmitting.
2. $t = \text{PROP} - \varepsilon$: A's first bit has *almost* reached B. B senses the line idle and starts its own packet.
3. $t = \text{PROP}$: A's bits reach B. B is receiving while transmitting, so **B detects the collision** (after only $\varepsilon$).
4. B's first bits travel back to A, arriving at $t = 2\,\text{PROP} - \varepsilon$. Only now does **A detect the collision**.

**MINIMUM FRAME DURATION (SLIDE 82)**

To be sure that a packet was sent without collision, a host must be able to detect a collision **before it finishes transmitting** the packet. In the worst case detection takes almost 2 PROP, so

$$\text{TRANSP} \;\geq\; 2 \cdot \text{PROP}.$$

If A had already finished at $t < 2\,\text{PROP}$, it would wrongly believe the packet had got through.

**EXAMPLE · WHY ETHERNET WAITS IN UNITS OF 512 BITS**

Classic 10 Mbps Ethernet uses 512 bit times as its basic unit (slide 91). At 10 Mbps one bit takes 0.1 $\mu$s, so 512 bits take 51.2 $\mu$s. A frame of at least 512 bits (64 bytes) therefore lasts at least 51.2 $\mu$s, which covers a round trip 2 PROP of up to 51.2 $\mu$s. At $2 \times 10^8$ m/s that is about 5 km of round trip, so about 2.5 km of cable end to end (in practice less, because repeaters add delay).

## > 13.2 The goodput of CSMA/CD (slides 83-88)

**DEF · EFFICIENCY $\eta$**

$$\eta = \frac{\text{time taken to send data}}{\text{time taken to send data} + \text{overhead}}.$$

**Model.** Time is slotted, and all packets have the same length. A **slot is 2 PROP**, the longest it can take to detect a collision. In each slot, each of $N$ hosts decides to transmit with probability $p$ (first transmissions and retransmissions alike).

1. **A slot is "won"** when exactly one host transmits. As in slotted ALOHA,

$$\alpha(p) = \binom{N}{1} p(1-p)^{N-1}, \qquad \alpha_{\max} = \left(1 - \tfrac{1}{N}\right)^{N-1} \text{ at } p = \tfrac{1}{N}.$$

2. **Wasted slots.** Let $A$ be the expected number of slots wasted before a packet gets through. In the first slot: with probability $\alpha$ we win and waste 0; with probability $1-\alpha$ we waste this slot and are back where we started, still expecting $A$ more:

$$A = \alpha \cdot 0 + (1-\alpha)(1+A) \quad \Rightarrow \quad A = \frac{1-\alpha}{\alpha}.$$

   This is exactly "failures before the first success" of a geometric variable.

3. **With** $\alpha = 0.4$: $A = 0.6/0.4 = 1.5$. Coin check (slide 85): a coin with $P(\text{heads}) = 0.4$ needs on average $1/0.4 = 2.5$ tosses, i.e. 1.5 failures and then 1 success.

4. **Efficiency.** Each packet costs its transmission time plus $A$ wasted slots of 2 PROP:

$$\eta_{\text{CSMA/CD}} = \frac{\text{TRANSP}}{\text{TRANSP} + A \cdot 2\,\text{PROP}} = \frac{\text{TRANSP}}{\text{TRANSP} + 3\,\text{PROP}} = \frac{1}{1+3a}, \qquad a \equiv \frac{\text{PROP}}{\text{TRANSP}}.$$

5. **More precise models and simulation** give $\eta_{\text{CSMA/CD}} \approx \dfrac{1}{1+5a}$.

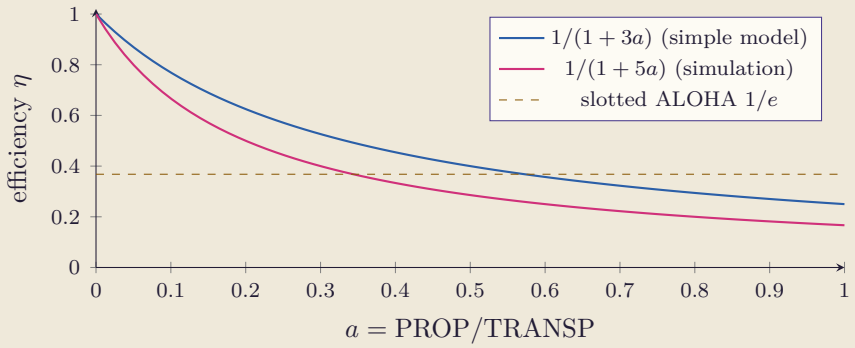
**PITFALL · SLIDE 84: "$\alpha_{\max} \approx 37\% \approx 40\%$"**

37% and 40% are not the same number. $\alpha_{\max} = (1 - \frac{1}{N})^{N-1}$ is 41% for $N = 5$ and falls to $1/e = 37\%$ only as $N \to \infty$. The slides then round to $\alpha = 0.4$ to get the neat $A = 1.5$. With the large-$N$ value $\alpha = 1/e$ we get $A = e - 1 \approx 1.72$ and $\eta = 1/(1 + 3.44a)$. The message is the same: the overhead is a small multiple of PROP per packet.

**EVERYTHING DEPENDS ON $a$ = PROP/TRANSP**

$a$ compares "how long it takes to find out about a collision" with "how long the useful work lasts".

- $a \to 0$ (short cable or long frames): $\eta \to 1$. A collision is noticed and aborted almost instantly.
- $a$ large (long distances, very fast links, short frames): $\eta$ collapses. This is why CSMA/CD only works in a *local* area.




CSMA/CD efficiency vs. $a$. For a LAN, $a$ is around 0.01, so $\eta$ is above 90%, far better than ALOHA's 37%.

SECTION 14

# Summary of MAC protocols

- **Channel partitioning** by time, frequency or code: TDMA, FDMA (and CDMA).
- **Taking turns**:
  - polling from a central site: a master node invites each node in turn to send up to some maximum number of frames;
  - token passing: a special "token" frame is passed from node to node, and only the holder may send.

  Examples: Bluetooth, FDDI, IBM Token Ring [KR 6.3.3].
- **Random access**: ALOHA, slotted ALOHA, CSMA, CSMA/CD. Carrier sensing is easy in some technologies (wire) and hard in others (wireless). CSMA/CD is used in Ethernet; CSMA/CA in 802.11 (WiFi).

## > 14.1 Scorecard against the wish list

| PROTOCOL | 1: ALONE $\to R$ | 2: $M$ BUSY $\to R/M$ | 3: DECENTR. | 4: NO SYNC | 5: SIMPLE |
|---|---|---|---|---|---|
| TDMA | no ($R/N$) | yes | yes* | no | yes |
| FDMA | no ($R/N$) | yes | yes* | yes | yes |
| Polling | almost | almost | no (master) | yes | fairly |
| Slotted ALOHA | yes | no ($\leq 0.37R$) | yes | no | yes |
| Pure ALOHA | yes | no ($\leq 0.18R$) | yes | yes | yes |
| CSMA/CD | yes | nearly ($\eta \approx \frac{1}{1+5a}$) | yes | yes | yes |

* once the slots or bands have been assigned. No protocol scores "yes" everywhere; random access gives up property 2 under heavy load in exchange for everything else.

SECTION 15

# Ethernet's CSMA/CD algorithm

**ETHERNET CSMA/CD, AS RUN BY THE NIC (SLIDE 90)**

1. The NIC receives a datagram from the network layer and creates a frame.
2. If the NIC senses the channel **idle**, it starts transmitting the frame. If it senses the channel **busy**, it waits until the channel is idle, then transmits.
3. If the NIC transmits the entire frame without detecting another transmission, it is done with the frame.
4. If the NIC detects another transmission while transmitting, it **aborts** and sends a jam signal.
5. After aborting, the NIC enters binary exponential backoff: after the $m$-th collision it chooses $K$ at random from $\{0, 1, 2, \ldots, 2^m - 1\}$, waits $K \cdot 512$ bit times, and returns to step 2.

**DEF · JAM SIGNAL AND BIT TIME**

- Jam signal: 48 bits sent after aborting, to make sure that *all* other transmitters notice the collision.
- Bit time: the time to send one bit: $0.1\,\mu$s at 10 Mbps (and $0.01\,\mu$s at 100 Mbps).

## > 15.1 Exponential backoff, step by step

**Goal**: adapt the retransmission attempts to the current load. Under heavy load, the random wait should be longer.

| COLLISION $m$ | $K$ CHOSEN FROM | MAX WAIT AT 10 MBPS | P(2 NODES PICK SAME $K$) |
|---|---|---|---|
| 1 | $\{0, 1\}$ | $51.2\,\mu$s | $1/2$ |
| 2 | $\{0, \ldots, 3\}$ | $153.6\,\mu$s | $1/4$ |
| 3 | $\{0, \ldots, 7\}$ | $358.4\,\mu$s | $1/8$ |
| 10 and up | $\{0, \ldots, 1023\}$ | $\approx 52.4$ ms | $1/1024$ |

- The wait for $K$ is $K \cdot 512 \cdot 0.1\,\mu\text{s} = K \cdot 51.2\,\mu$s. For $K = 1023$: $1023 \times 51.2\,\mu\text{s} \approx 52.4$ ms, the slide's "about 50 msec".
- The last column is for *two* colliding nodes: they collide again only if they pick the same $K$, which happens with probability $1/2^m$.

**WHY "EXPONENTIAL" IS THE RIGHT SHAPE** [KR 6.3.2]

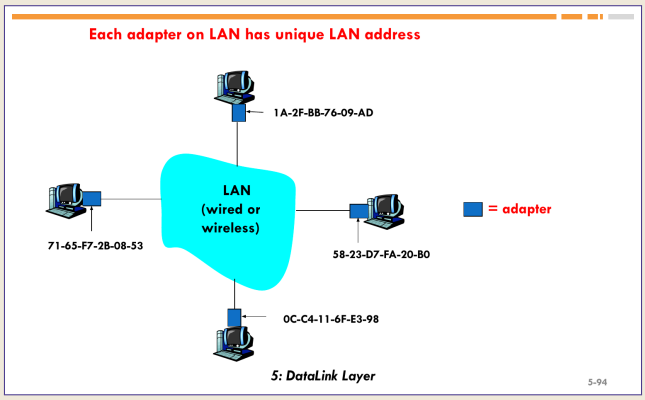
We want the interval to be **short when few nodes collide** (so the channel is not left idle) and **long when many collide** (so their random choices spread out). Nobody knows how many nodes are colliding, but each new collision is evidence that there are many, so the interval doubles. The doubling stops at $m = 10$.

SECTION 16

# MAC addresses

**DEF · MAC ADDRESS**

A MAC address (also LAN, physical or Ethernet address) is used to get a frame from one interface to another *physically connected* interface on the same network. For most LANs it is **48 bits** long, written as 6 bytes in hexadecimal, e.g. `1A-2F-BB-76-09-AD`. It is burned into the NIC's ROM, and is sometimes also settable in software.




Slide 94. Each adapter (blue) on the LAN has its own unique MAC address. Addresses belong to adapters, not to hosts: a host with two NICs has two MAC addresses [KR 6.4.1].

**WHO HANDS OUT MAC ADDRESSES**

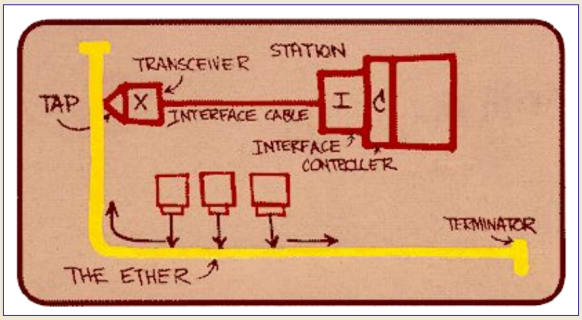
The IEEE administers the address space. A manufacturer buys a block of addresses: the IEEE fixes the first 24 bits, and the manufacturer assigns the last 24 bits, which is $2^{24} \approx 16.8$ million unique addresses per block [KR 6.4.1]. In total there are $2^{48} \approx 2.8 \times 10^{14}$ addresses.

**FLAT VS. HIERARCHICAL: ID NUMBER VS. POSTAL ADDRESS**

A MAC address is like your **ID number** (*teudat zehut*): a flat identifier that says nothing about where you are, and stays the same wherever you go. So a network card can be moved from one LAN to another and keep its address: flat ⇒ portable. An IP address (later in the course) is hierarchical, like a postal address: it encodes *where* you are, so it must change when you move, and it is not portable.

SECTION 17

# Ethernet




Bob Metcalfe's original Ethernet sketch (slide 97): stations attached through transceivers to one shared cable, "the ether".

**DEF · ETHERNET**

The "dominant" wired LAN technology: cheap (about $20 for a NIC); the first widely used LAN technology; simpler and cheaper than token LANs and ATM; it kept up with the speed race, from 10 Mbps to 100 Gbps. Metcalfe created it by modifying the ALOHA protocol into CSMA/CD [KR 6.3.2].

**ETHERNET GOODPUT (SLIDE 98)**

Let $t_{\text{prop}}$ = the maximum propagation delay between two nodes in the LAN, and $t_{\text{trans}}$ = the time to transmit a maximum-size frame. Then

$$\text{goodput} \approx \frac{1}{1 + 5\, t_{\text{prop}}/t_{\text{trans}}}.$$

Goodput tends to 1 as $t_{\text{prop}} \to 0$ and as $t_{\text{trans}} \to \infty$. This is the $1/(1+5a)$ of CSMA/CD with $a = t_{\text{prop}}/t_{\text{trans}}$.

**EXAMPLE · A 10 MBPS ETHERNET WITH NUMBERS**

Maximum frame $\approx$ 1518 bytes = 12,144 bits, so $t_{\text{trans}} = 12{,}144/10^7 \approx 1.21$ ms. A 500 m cable at $2 \times 10^8$ m/s: $t_{\text{prop}} = 2.5\,\mu$s. Then $a = 2.5/1214 \approx 0.0021$ and goodput $\approx 1/(1 + 0.0103) \approx 99\%$.

Now speed the same LAN up to 1 Gbps: $t_{\text{trans}} \approx 12.1\,\mu$s, $a \approx 0.21$, goodput $\approx 1/(1{+}1.03) \approx 49\%$. Faster links shrink $t_{\text{trans}}$ and make the shared-bus design worse, one reason today's Ethernet uses switched stars.

SECTION 18

# WiFi (IEEE 802.11): CSMA/CA

WHY WIFI CANNOT USE COLLISION DETECTION (SLIDE 99)

802.11 does carrier sensing (it does not start while another node is transmitting), but it has **no collision detection**:

- it is hard to receive (sense a collision) while transmitting, because the received signals are weak compared to the node's own (fading);
- even then, it could not sense all collisions: the **hidden terminal** problem and fading mean the collision happens at the receiver, out of the sender's hearing.

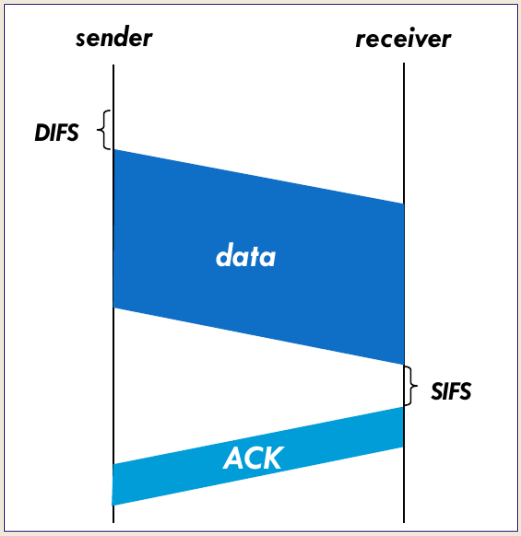
So the goal changes: **avoid** collisions. Hence CSMA/C(ollision)A(voidance).

DEF · 802.11 CSMA/CA (SLIDE 100)

**Sender**:

1. If it senses the channel idle for a period called DIFS (distributed inter-frame space), it transmits the entire frame (there is no CD, so there is no aborting).
2. If it senses the channel busy, it starts a **random backoff** timer. The timer counts down only while the channel is idle (it is frozen while busy). The node transmits when the timer expires. If no ACK comes back, it increases the random backoff interval and repeats step 2.

**Receiver**: if the frame is received OK, it returns an ACK after a short period SIFS (short inter-frame space).




Slide 100. The sender waits DIFS, sends the data; the receiver waits SIFS and sends the ACK.

WHY ACKS, AND WHY BACK OFF EVEN AFTER AN IDLE CHANNEL [KR 7.3.2]

- **ACKs**: without collision detection, the sender cannot know whether its frame arrived. Wireless links also have high bit-error rates. The link-layer ACK is the only evidence.
- **Backoff**: suppose two stations wait for a third to finish. With Ethernet's rule they would both start the moment it ends, and collide, which is cheap in Ethernet because they abort. In WiFi a collision costs the *whole* frame. So both draw random backoffs instead; the one with the smaller value goes first, and the other hears it and freezes its counter.

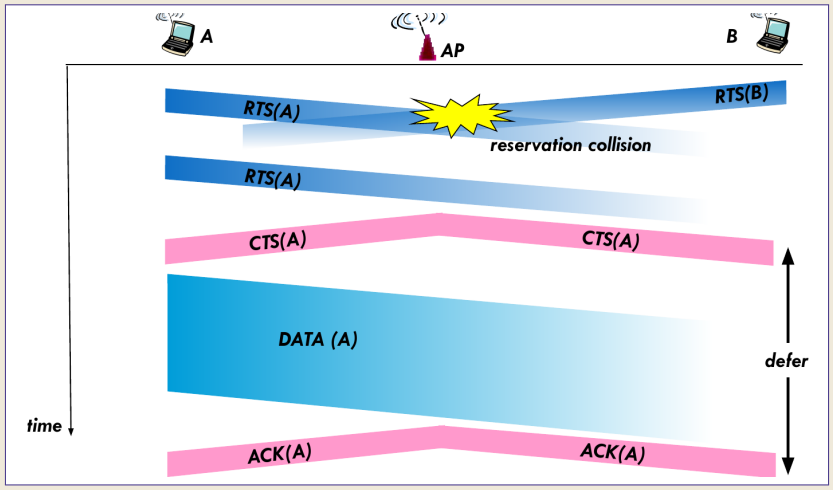
## > 18.1 Reserving the channel: RTS and CTS

DEF · RTS/CTS (SLIDE 101)

Idea: let the sender **reserve** the channel instead of sending long data frames at random.

1. The sender first transmits a small request-to-send (RTS) packet to the access point (AP), using CSMA. RTSs may still collide with each other, but they are short, so little time is lost.
2. The AP broadcasts a clear-to-send (CTS) in response to the RTS.
3. **Every node hears the CTS**, including nodes hidden from the sender. The sender transmits its data frame, and the other stations defer.

Collisions of data frames are avoided completely, using small reservation packets.




Slide 102. A and B send RTS at the same time: a *reservation collision* (cheap, the RTS is short). A retries; the AP answers CTS(A), heard by both A and B. A sends DATA(A); B defers until the ACK(A).

EXAMPLE · WHY RTS/CTS SOLVES THE HIDDEN TERMINAL

Take slide 43's picture: A and C are hidden from each other behind the mountain, and both reach B, the AP.

1. A sends an RTS to B. C does not hear it.
2. B broadcasts CTS(A). **C hears the CTS**, because C does reach B.

3. C now knows the channel is reserved, and stays quiet until it hears B's ACK(A).

Carrier sensing failed because C could not hear A, the *sender*. RTS/CTS works because C can hear B, the *receiver*, which is exactly where collisions happen.

EXAMPLE · IS THE RESERVATION WORTH IT?

Say RTS and CTS take 20 $\mu$s each and a data frame takes 1000 $\mu$s. A data-frame collision wastes 1000 $\mu$s; an RTS collision wastes only 20 $\mu$s. The price is about 40 $\mu$s of extra control per frame, 4%. It pays off when collisions are likely. It does not pay off for very short frames, where the RTS/CTS would be as long as the data.

SECTION 19

# Summary: MAC protocols in practice

WEEK 02 IN ONE PARAGRAPH

Networks are built as a stack of five layers: physical, link, network, transport, application. Each layer talks only to its neighbours and its peers, and adds its own header (encapsulation). IP is the single narrow waist. Building bottom up, the first challenge is a LAN: a shared broadcast link needs a MAC protocol. **Channel partitioning** (TDMA, FDMA) is fair but wastes the channel when few nodes are busy. **Random access** gives a lone node the full rate but loses to collisions: slotted ALOHA at most $1/e \approx 37\%$, pure ALOHA $1/(2e) \approx 18\%$. Listening first (CSMA) and aborting early (CSMA/CD, *Ethernet*) push the efficiency to $1/(1+5a)$, close to 100% in a LAN. Wireless cannot detect collisions, so *802.11* avoids them (CSMA/CA, ACKs, RTS/CTS).

PART B · RECITATION 2

# ALOHA, Step by Step

**What the recitation adds.** Recitation 2 ("ALOHA", 50 slides, 19 October 2026) revisits the random access part of the lecture, but from the exam's point of view. Parts of it repeat Part A (slotted and pure ALOHA). The new material is:

- the 7-layer **OSI model** (the lecture used 5 layers);
- precise definitions of **bandwidth, throughput and goodput**, with two warm-up calculations;
- **two ways** to compute the goodput (one slot vs. waiting for a success) and the **binomial vs. Poisson** models;
- two exam-style questions that **mix** slotted and pure ALOHA in one network.

Every slide question is solved in full below, then extended with numbers. The recitation announces a Poisson analysis but never does it; §27 fills it in, following Bertsekas & Gallager [BG 4.2.2, 4.2.4].

SECTION 20

# The OSI model: seven layers

The lecture built the Internet's *five* layers (Part A, §Layering). The recitation starts from the older reference model with *seven*.

DEF · OSI MODEL

The Open Systems Interconnection (OSI) model is an abstract description of the functions a communication system needs, split into layers. As in Part A, each layer uses the service of the layer beneath it and serves the layer above it. OSI has 7 layers; the Internet stack merges three of them into one and has 5.

| # | LAYER | DATA UNIT | FUNCTION |
|---|---|---|---|
| 7 | Application | data | high-level APIs between applications (e.g. an HTTP GET) |
| 6 | Presentation | data | translating data between the network and the application (e.g. encryption, compression) |
| 5 | Session | data | managing a continuing session (setting it up, checkpointing, ending it) |
| 4 | Transport | segment | delivery between processes, reliable or not (acknowledgements) |
| 3 | Network | packet | multi-hop networks: global addressing, routing |
| 2 | Data link | frame | delivering frames between two neighbouring nodes |
| 1 | Physical | bit | sending raw bits over a medium (copper, fibre, air) |

WHERE DID LAYERS 5 AND 6 GO?

The Internet stack has no separate session or presentation layer. If an application needs these services (encryption, for example, through TLS), the application provides them itself. That is why the Internet has 5 layers: OSI layers 5–7 together form the Internet's application layer.

EXAMPLE · WHICH LAYER? (SLIDE 6)

DNS, HTTP → layer 7. TCP, UDP → layer 4. IP → layer 3. MAC protocols (ALOHA, CSMA/CD) → layer 2. The radio or the cable → layer 1.

PITFALL · SLIDE 6: "LAYER 1: WI-FI", "LAYER 2: MAC"

**Wi-Fi is not just a layer 1 protocol.** IEEE 802.11 specifies *both* the physical layer (radio) *and* the link layer, including its MAC protocol, CSMA/CA (Part A, §WiFi). Ethernet is

likewise both layers 1 and 2. And "MAC" is not one protocol but a *sub-layer* of layer 2: the part that decides who may transmit. ALOHA, CSMA/CD and CSMA/CA are all MAC protocols.

SECTION 21

# Random access: the setting

DEF · SHARED MEDIUM

In a shared medium, all nodes transmit on, and listen to, the same channel. The recitation's rules:

1. a node that transmits uses the full bandwidth of the link;
2. if two or more nodes transmit at the same time, there is a collision and all the data involved is lost;
3. noise is ignored: a frame that does not collide arrives intact.




After slide 8.

SECTION 22

# Bandwidth, throughput and goodput

DEF · BANDWIDTH $B$

The bandwidth $B$ is the maximum rate at which the channel can carry data, in bits per second. A frame of $L$ bits therefore takes $T = L/B$ seconds to send. (Kurose & Ross call the same quantity the transmission rate $R$ [KR 1.4.1].)

DEF · THROUGHPUT AND GOODPUT (RECITATION'S DEFINITIONS)

- Throughput: the fraction of the bandwidth (equivalently, of the time) that is used to send *anything*, including frames that collide.
- Goodput $\eta$: the fraction of the bandwidth used to send traffic *successfully*:

$$\eta = \frac{\text{average effective bandwidth used}}{\text{total bandwidth}} = \frac{\text{average time of successful transmissions}}{\text{total time}},$$

with $0 \le \eta \le 1$.

The two forms agree because a transmitting node always uses the full bandwidth. Here both

quantities are unitless fractions.

**A NOISY CLASSROOM (SLIDE 9)**

In a class where everybody talks at once, the air is busy all the time: **throughput is high**. But nobody can follow anything: **goodput is close to zero**. A random access protocol is judged by its goodput.

**PITFALL · THROUGHPUT HAS TWO MEANINGS**

In the recitation, throughput is a unitless *fraction*. In Kurose & Ross and in most of the course, throughput is a *rate* in bits/s ("the rate at which the receiver gets bits", [KR 1.4.4]). The two are linked by multiplying by $B$: a goodput of 0.37 on a 100 Mbps link is a useful rate of 37 Mbps. Always check the units in a question.

**EXAMPLE · SLIDE 10, QUESTION 1: TIME TO SEND A PACKET**

$B = 5$ bit/s, packet size 30 bits. $T = \dfrac{|\text{packet}|}{B} = \dfrac{30 \text{ bit}}{5 \text{ bit/s}} = 6 \text{ s}.$

**EXAMPLE · SLIDE 10, QUESTION 2: BANDWIDTH NEEDED**

We need to send 3 packets of 30 bits within 2 s. In total that is 90 bits, so $B = \dfrac{3 \cdot 30 \text{ bit}}{2 \text{ s}} = 45$ bit/s.

**EXAMPLE · EXTENSION: THE SAME QUESTIONS WITH REAL NUMBERS**

A 1500-byte Ethernet frame (12 000 bits) on a 100 Mbps link takes $T = 12\,000/10^8 = 120\,\mu$s. Streaming 1000 such frames per second needs $B \geq 1000 \cdot 12\,000 = 12$ Mbps. With a random access protocol of goodput $\eta$, the link must be faster still: $B \geq 12/\eta$ Mbps, i.e. about 32.6 Mbps for slotted ALOHA at its best ($\eta = 1/e$).

SECTION 23

# ALOHA: a brief history

ALOHAnet was Norman Abramson's radio packet network at the University of Hawaii (the slides date the project to 1968; it was described in 1970) [KR 6.3.2, case history]. Laying cables between mountainous islands was impractical, so terminals scattered over the islands talked to one central host by radio, on **two frequencies**:

- **downlink**, central host → terminals: only the host transmits, so there is no conflict. The host also sends an acknowledgement for every packet it received;
- **uplink**, terminals → host: shared by all terminals, so their packets can collide.

The designers considered dividing the uplink (multiplexing, Part A, §TDMA/FDMA), but chose instead to let every terminal simply transmit, and retransmit after a random delay if no acknowledgement came back. That is (pure) ALOHA, the first multiple access protocol. Its random access idea inspired Bob Metcalfe's CSMA/CD and Ethernet (Part A).

SECTION 24

# The general model

DEF · THE RECITATION'S ALOHA MODEL (SLIDES 16-17)

- $N$ nodes; usually we look at the limit of many nodes, $N \to \infty$.
- Bandwidth $B$; every frame has $L$ bits. Time is divided into slots of length $T = L/B$: one slot = the time to send one frame.
- A successful transmission uses the entire bandwidth for the entire slot.
- A node may start only at the beginning of a slot; a message that arrives mid-slot waits for the next slot.
- When a node finishes transmitting, it knows whether there was a collision; if so, it resends later.
- Propagation delay is ignored.

DEF · SLOTTED VS. PURE ALOHA (SLIDE 18)

- Slotted ALOHA: all nodes' clocks are synchronized, so all slots are aligned.
- Pure ALOHA: each node has its *own* clock (synchronizing clocks is expensive). Each node still uses slots of length $T$, but different nodes' slots are shifted against each other.

DEF · BINOMIAL VS. POISSON MODELS (SLIDE 19)

Let $X$ be the number of transmissions that start in one slot.

- Binomial model: $n$ nodes, each sending with probability $p$, independently. Then $X \sim \mathrm{Bin}(n, p)$.
- Poisson model: infinitely many nodes, which together start on average $G$ transmissions per slot. Then $X \sim \mathrm{Poi}(G)$.

The two are linked by the Poisson approximation from Week 01: if $n$ is large and $p$ small, $\mathrm{Bin}(n, p) \approx \mathrm{Poi}(np)$. So $G = np$.

PITFALL · SLIDE 19: "AT A RATE OF $\lambda$ PER $T$" AND $X \sim \mathrm{Poi}(\lambda T)$

These two statements do not fit together. If $\lambda$ counts transmissions *per slot*, then $X \sim \mathrm{Poi}(\lambda)$. If $\lambda$ counts transmissions *per second*, then the mean in a slot of $T$ seconds is $\lambda T$, and $X \sim \mathrm{Poi}(\lambda T)$. The second reading is the usual one. To avoid the clash, we write $G = \lambda T$ for the mean number per slot (Bertsekas & Gallager's notation, [BG 4.2.2]).

SECTION 25

# Slotted ALOHA, binomial model

**Assumptions (slide 22).** All $n$ nodes always have something to send; in every slot, each node transmits with probability $p$, independently. This is the same simplified model as in Part A. A slot is successful if exactly one node transmits.

## > 25.1 View 1: look at one slot (slide 24)

1. **All slots are alike**, so the long-run fraction of useful time equals what happens in *one* typical slot.
2. **Count transmissions**: $X \sim \mathrm{Bin}(n, p)$, so $P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}$.
3. **Success** means $X = 1$:

$$P_{\mathrm{suc}} = P(X = 1) = \binom{n}{1} p(1-p)^{n-1} = np(1-p)^{n-1}.$$

4. **Useful time in the slot**: let $T_{\mathrm{suc}}$ be the time spent on successful transmission in the slot. It is $T$ if the slot succeeds and 0 otherwise, so

$$\mathbb{E}[T_{\mathrm{suc}}] = T \cdot P_{\mathrm{suc}} + 0 \cdot (1 - P_{\mathrm{suc}}) = T\, P_{\mathrm{suc}}.$$

5. **Goodput**: useful time over total time,

$$\boxed{\eta = \frac{\mathbb{E}[T_{\mathrm{suc}}]}{T} = P_{\mathrm{suc}} = np(1-p)^{n-1}}$$

## > 25.2 View 2: wait for a success (slide 25)

1. Each slot succeeds with probability $P_{\text{suc}}$, independently of the other slots.
2. The number of slots up to and including the first success is geometric: $X \sim \text{Geo}(P_{\text{suc}})$, with $\mathbb{E}[X] = 1/P_{\text{suc}}$.
3. So on average, every $1/P_{\text{suc}}$ slots (time $T/P_{\text{suc}}$) deliver one frame (useful time $T$):

$$\eta = \frac{T}{T \cdot \frac{1}{P_{\text{suc}}}} = P_{\text{suc}}.$$

**SAME ANSWER, TWO STORIES**

View 1 asks "what fraction of each slot is useful, on average?". View 2 asks "how long do I wait for each useful slot?". A rate (successes per slot) and a waiting time (slots per success) are reciprocals of each other, so both give $\eta = P_{\text{suc}}$. Use whichever is easier in a question.

**EXAMPLE · $n = 10$ NODES, $p = 0.05$**

$P_{\text{suc}} = 10 \cdot 0.05 \cdot 0.95^9 = 0.5 \cdot 0.630 = 0.315$. View 2: a success every $1/0.315 \approx 3.2$ slots on average. With $T = 1\,\text{ms}$, that is one delivered frame every 3.2 ms, and a goodput of 31.5%.

## > 25.3 The best $p$ and the limit (slides 26-27)

Part A (§The goodput of slotted ALOHA) carries out the derivative: $\frac{d\eta}{dp} = n(1-p)^{n-2}[(1-p) - (n-1)p] = 0$ gives $p^* = 1/n$. The bracket is positive for $p < 1/n$ and negative for $p > 1/n$, so this is a maximum. Then

$$\eta(p^*) = \left(1 - \frac{1}{n}\right)^{n-1} \xrightarrow{n\to\infty} \frac{1}{e} \approx 0.37.$$

**EXAMPLE · HOW SHARP IS THE PEAK? $n = 10$**

| $p$ | 0.02 | 0.05 | 0.08 | **0.10** | 0.12 | 0.15 | 0.20 |
|---|---|---|---|---|---|---|---|
| $\eta = 10p(1-p)^9$ | 0.167 | 0.315 | 0.378 | **0.387** | 0.380 | 0.347 | 0.268 |

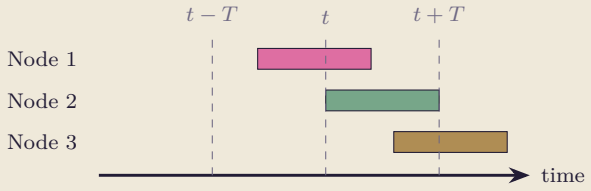
Getting $p$ within $\pm 20\%$ of $1/n$ costs little. Being off by a factor of 2 costs a lot, and in a real network the nodes do not know $n$, which makes this a real problem.

**WHY ALOHA IS STILL USED**

Slotted ALOHA is completely distributed: no node needs to know about the others, apart from the slot clock. The price is that at best 37% of the channel is used (slide 27).

SECTION 26

# Pure ALOHA, binomial model




After slide 29. Node 2 starts at time $t$. Node 1 started less than $T$ earlier and node 3 starts less than $T$ later: both overlap node 2's frame, so all three frames are lost.

DEF · VULNERABLE PERIOD

The vulnerable period of a frame is the set of times at which *another* transmission may not start, or the frame collides.

- Slotted ALOHA: only the frame's own slot, length $T$.
- Pure ALOHA: any start in $(t-T,\, t+T)$ overlaps a frame sent at $t$, so the vulnerable period is $2T$.

**The success probability, step by step.**

1. Fix a node $i$ that transmits in its own slot $[t, t+T]$: probability $p$.
2. Any other node $j$ has its own, shifted slots. Exactly **two** of node $j$'s slots overlap $[t, t+T]$: the one that started in $(t-T, t]$ and the one that starts in $(t, t+T)$.
3. Node $j$ stays silent in both with probability $(1-p)^2$. For all $n-1$ other nodes, by independence: $\left((1-p)^2\right)^{n-1} = (1-p)^{2(n-1)}$.
4. So node $i$ succeeds with probability $p(1-p)^{2(n-1)}$. The success events of different nodes are disjoint, so we add over the $n$ nodes:

$$\boxed{\eta = P_{\text{suc}} = np(1-p)^{2(n-1)}}$$

(The goodput step is the same as in view 1 above: $\mathbb{E}[T_{\text{suc}}] = T\,P_{\text{suc}}$.)

PITFALL · SLIDE 30: $P(\cdot) \cap P(\cdot)$

Slide 30 writes

$$P_{\text{suc}} = P(\text{some node transmitted}) \cap P(\text{no other node transmitted in two overlapping slots}).$$

Two problems:

- $\cap$ is an operation on *events*, not on numbers. The correct form is $P(A \cap B)$, which equals $P(A)P(B)$ when $A$ and $B$ are independent (the next line of the slide).
- "$P$(some node transmitted)" is then set to $np$. But the probability that *some* node transmits is $1-(1-p)^n$, not $np$ (for $n=10$, $p=0.2$: 0.89, while $np=2$, not even a probability). The factor $n$ really comes from *adding* the $n$ disjoint events "node $i$ transmits and all others are

silent", as in step 4.

The final formula on the slide is correct.

**The best** $p$ (slide 31 only states it). Differentiate as in the slotted case:

$$\frac{d\eta}{dp} = n(1-p)^{2n-3}\big[(1-p) - 2(n-1)p\big] = 0 \quad \Longrightarrow \quad p^* = \frac{1}{2n-1}.$$

Substituting, with $m = 2n-1$:

$$\eta(p^*) = \frac{n}{2n-1}\Big(1 - \frac{1}{m}\Big)^{m-1} \xrightarrow{n\to\infty} \frac{1}{2}\cdot\frac{1}{e} = \frac{1}{2e} \approx 0.18.$$

PITFALL · SLIDE 31: "THE GOODPUT IS $1/(2e)$"

$1/(2e)$ is the goodput only in the limit of many nodes. For finite $n$ the best goodput is larger: 0.296 for $n = 2$, 0.216 for $n = 5$, 0.191 for $n = 20$ (table in Part A).

SECTION 27

# The Poisson models

Slide 20 promises four models, but the recitation analyses only the two binomial ones. Here are the other two. Now infinitely many nodes start on average $G$ transmissions per slot, so the number of starts in any interval of length $\tau$ is $\mathrm{Poi}(G\tau/T)$ [BG 4.2.2]. Recall $P(\mathrm{Poi}(\mu) = k) = e^{-\mu}\mu^k/k!$.

**Slotted, Poisson.**

1. A slot succeeds iff exactly one transmission starts in it: $P(X = 1) = e^{-G}G$.
2. Goodput, as in view 1: $\eta = Ge^{-G}$.
3. Best $G$: $\frac{d}{dG}Ge^{-G} = (1-G)e^{-G} = 0$, so $G^* = 1$ and $\eta_{\max} = e^{-1} \approx 0.37$.

**Pure, Poisson.**

1. A frame starting at $t$ succeeds iff no other frame starts in its vulnerable period of length $2T$. The number of starts there is $\mathrm{Poi}(2G)$, so $P(\text{no other start}) = e^{-2G}$.
2. Frames start at rate $G$ per slot, and each succeeds with probability $e^{-2G}$: $\eta = Ge^{-2G}$ [BG 4.2.4].
3. Best $G$: $(1-2G)e^{-2G} = 0$, so $G^* = \frac{1}{2}$ and $\eta_{\max} = \frac{1}{2}e^{-1} = \frac{1}{2e} \approx 0.18$.

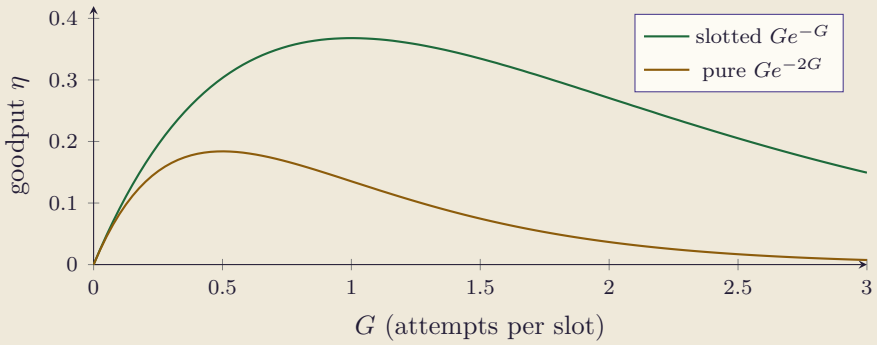
EXAMPLE · GOODPUT VS. LOAD $G$

| $G$ (ATTEMPTS PER SLOT) | SLOTTED $Ge^{-G}$ | PURE $Ge^{-2G}$ |
|---|---|---|
| 0.25 | 0.195 | 0.152 |
| 0.5 | 0.303 | **0.184** |
| 1 | **0.368** | 0.135 |
| 2 | 0.271 | 0.037 |

At $G = 2$ the channel is overloaded: slotted ALOHA keeps 27%, pure ALOHA only 4%.

**THE TWO MODELS AGREE**

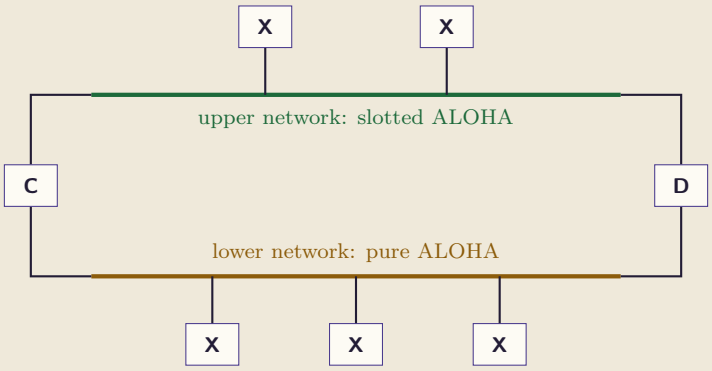
Put $p = G/n$ in the binomial formula: $np(1-p)^{n-1} = G(1-G/n)^{n-1} \to Ge^{-G}$ as $n \to \infty$, the same limit as in Part A's $(1-1/N)^N \to 1/e$. The Poisson model is simply the binomial model with infinitely many nodes, each very quiet. That is why both give $1/e$ and $1/(2e)$.




Poisson models. The slotted curve peaks at $G = 1$ with $1/e$, the pure one at $G = 1/2$ with $1/(2e)$. Beyond the peak, more attempts mean *less* goodput.

SECTION 28

# Question 1: one host, two networks




After slide 32. C and D are attached to both networks. The upper network (slotted ALOHA) has two other nodes, the lower one (pure ALOHA) has three.

**Setup (slide 33).** D never sends. When C has a message for D, it picks the upper network with probability $p$ (the lower one with probability $1-p$) and then sends with probability $q$. In every slot, each X node sends with probability $q$, independently.

## Question 1a: a single attempt

**Question.** C has one attempt. What is the probability that the message reaches D?

1. **Split by the network chosen** (law of total probability):

$$P_{\text{suc}}^{\text{total}} = p \cdot P_{\text{suc}}^{\text{upper}} + (1-p) \cdot P_{\text{suc}}^{\text{lower}}.$$

2. **Upper network (slotted)**. C must send ($q$) and the two X nodes must be silent *in this one slot* ($(1-q)^2$):

$$P_{\text{suc}}^{\text{upper}} = q(1-q)^2.$$

3. **Lower network (pure)**. C must send ($q$), and each of the three X nodes must be silent in *both* of its slots that overlap C's frame ($(1-q)^2$ each):

$$P_{\text{suc}}^{\text{lower}} = q\left((1-q)^2\right)^3 = q(1-q)^6.$$

4. **Combine**:

$$\boxed{P_{\text{suc}}^{\text{total}} = q\left[p(1-q)^2 + (1-p)(1-q)^6\right]}$$

**EXAMPLE · NUMBERS: $p = 0.5$, $q = 0.2$**

Upper: $0.2 \cdot 0.8^2 = 0.128$. Lower: $0.2 \cdot 0.8^6 = 0.0524$. Total: $0.5 \cdot 0.128 + 0.5 \cdot 0.0524 = 0.090$. The lower network is worse for two reasons: it has one more competing node, and pure ALOHA doubles each node's vulnerable window. Each X node there counts like *two* slotted nodes, so the lower network behaves like a slotted network with six competitors.

**PITFALL · C'S OWN $q$**

C's chance of *sending at all* is $q$, so even an empty network gives success only with probability $q$. That is why $q$ multiplies everything. The best choice of network is the upper one for every $q$, since $(1-q)^2 \geq (1-q)^6$; so C should take $p = 1$.

## Question 1b: the goodput of the upper network

**Question.** C now tries at the beginning of every slot. What is the goodput of the upper network?

Now C is an active node like the others. In each slot it picks a network (upper with probability $p$) and sends with probability $q$. We need the probability that *some* frame on the upper network succeeds.

1. **C chose the upper network** (probability $p$): there are 3 active nodes (C and the two X's), each sending with probability $q$. Exactly one must send:

$$P_{\text{suc}}^{1} = \binom{3}{1} q(1-q)^2 = 3q(1-q)^2.$$

2. **C chose the lower network** (probability $1-p$): only the 2 X nodes compete on the upper network:

$$P_{\text{suc}}^{2} = \binom{2}{1} q(1-q) = 2q(1-q).$$

3. **Goodput** = probability of a successful slot (view 1):

$$\boxed{\eta^{\text{upper}} = p \cdot 3q(1-q)^2 + (1-p) \cdot 2q(1-q)}$$

**EXAMPLE · NUMBERS: $p = 0.5$, $q = 0.2$**

$3 \cdot 0.2 \cdot 0.64 = 0.384$ and $2 \cdot 0.2 \cdot 0.8 = 0.32$, so $\eta^{\text{upper}} = 0.5 \cdot 0.384 + 0.5 \cdot 0.32 = 0.352$. Compare with the slotted formula $nq(1-q)^{n-1}$: with $n = 3$ nodes the best $q$ is $1/3$ (goodput $4/9 = 0.444$); with $n = 2$ it is $1/2$ (goodput 0.5). $q = 0.2$ is too timid for such a small network.

**WHY 1B COUNTS ALL NODES AND 1A COUNTS ONLY C**

In 1a we asked whether *C's* message arrives: one specific node. In 1b we ask about the *network's* goodput: a slot is useful if *any* single node succeeds, so we add over the nodes, hence the binomial coefficients $\binom{3}{1}$ and $\binom{2}{1}$.

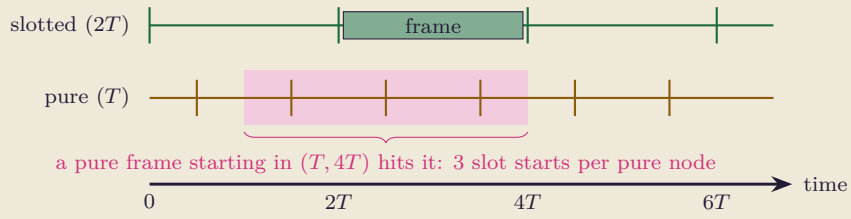
SECTION 29

# Question 2: slotted and pure nodes on one channel

**Setup (slide 42).** There are $2n$ nodes on one channel.

- Nodes $1, \ldots, n$ run **pure ALOHA** with slots of length $T$ (each on its own clock).
- Nodes $n+1, \ldots, 2n$ run **slotted ALOHA** with slots of length $2T$ (synchronized among themselves).

At the beginning of each of its slots, every node sends with probability $p$, and its message fills the whole slot. So pure nodes send short frames ($T$) and slotted nodes long ones ($2T$).




Question 2a. A slotted node's frame occupies $[2T, 4T]$. A pure node's frame of length $T$ collides with it if it starts anywhere in $(T, 4T)$, an interval of length $3T$ that contains exactly three of that pure node's slot starts.

## Question 2a: a slotted node

**Question.** In a random slot, what is the probability that a node running slotted ALOHA sends successfully?

A slotted node succeeds iff *all three* of these hold (they involve different nodes, so they are independent and we multiply):

1. it sends: $p$;
2. the other $n-1$ slotted nodes do not send in this $2T$ slot: $(1-p)^{n-1}$;
3. no pure node starts a frame in the $3T$ window of the figure. Each pure node has 3 slot starts there and must skip all three: $(1-p)^3$; for $n$ pure nodes, $(1-p)^{3n}$.

$$\boxed{P_{\text{suc}}^{\text{slotted}} = p\,(1-p)^{n-1}(1-p)^{3n}}$$

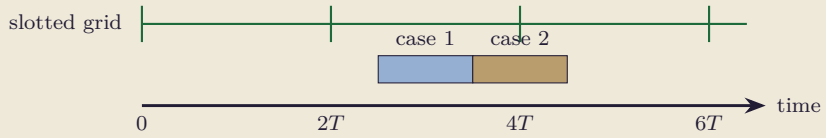
**WHY 3 AND NOT 2**

For two frames of the same length $T$, the vulnerable window is $2T$ (frame length plus the length of the other frame). In general it is (own length) + (other's length): here $2T + T = 3T$. Long frames are especially exposed to unsynchronized short ones.

## Question 2b: a pure node

**Question.** In a random slot, what is the probability that a node running pure ALOHA sends successfully?

The pure node's frame has length $T$. The other pure nodes are handled as in plain pure ALOHA: $(1-p)^{2(n-1)}$. The slotted nodes depend on where the frame falls relative to the $2T$ grid. There are two cases (slides 44–46):




Case 1: the pure frame lies inside one $2T$ slot. Case 2: it straddles the boundary between two $2T$ slots.

**Case 1, inside one $2T$ slot**: no slotted node may send in that one slot, $(1-p)^n$:

$$P_1 = p\,(1-p)^n (1-p)^{2(n-1)}.$$

**Case 2, straddling two $2T$ slots**: no slotted node may send in either, $(1-p)^{2n}$:

$$P_2 = p\,(1-p)^{2n} (1-p)^{2(n-1)}.$$

**Combining the cases** (the slides stop before this). Shift the pure node's slots by any offset $0 < \varphi < T$ against the $2T$ grid: its slots then alternate between case 1 and case 2. So exactly half of its slots are of each kind, and a random slot gives

$$\boxed{P_{\text{suc}}^{\text{pure}} = \tfrac{1}{2}P_1 + \tfrac{1}{2}P_2 = \tfrac{1}{2}\,p\,(1-p)^{2(n-1)}\left[(1-p)^n + (1-p)^{2n}\right]}$$

**EXAMPLE · NUMBERS: $n = 5$, $p = 0.1$**

Slotted node (2a): $0.1 \cdot 0.9^4 \cdot 0.9^{15} = 0.0135$. Pure node (2b): $P_1 = 0.1 \cdot 0.9^5 \cdot 0.9^8 = 0.0254$, $P_2 = 0.1 \cdot 0.9^{10} \cdot 0.9^8 = 0.0150$, average 0.0202. A pure node succeeds more often than a slotted one: its short frame has a smaller vulnerable window.

## Question 2c: everybody synchronized

**Question.** The pure nodes switch to slotted ALOHA with slots of length $T$, synchronized with the $2T$ slots. What is the goodput of the network?

Now every $2T$ slot of the long-frame nodes is split into two aligned $T$ slots of the short-frame nodes. Look at one $2T$ window. Write

$$s = np(1-p)^{n-1} \quad \text{(exactly one of } n \text{ nodes sends)}, \qquad z = (1-p)^n \quad \text{(none of } n \text{ nodes sends)}.$$

The window can contain these successful outcomes (slides 47–49):

| | OUTCOME | PROBABILITY | USEFUL TIME |
|---|---|---|---|
| A | one long frame succeeds: exactly one long node sends, no short node sends in either half | $s\,z^2$ | $2T$ |
| B | short success in the first half only: no long node sends, exactly one short node in half 1, not exactly one in half 2 | $s\,z\,(1-s)$ | $T$ |
| C | short success in the second half only (same as B) | $s\,z\,(1-s)$ | $T$ |
| D | short successes in both halves: no long node, exactly one short node in each half | $s^2 z$ | $2T$ |

Dividing useful time by $2T$ gives the slide's answer:

$$\eta = A + \tfrac{1}{2}B + \tfrac{1}{2}C + D.$$

**Simplifying.** Substitute and collect: $\eta = sz^2 + sz(1-s) + s^2 z = sz^2 + sz$, so

$$\boxed{\eta = s\,z\,(1+z) = np(1-p)^{2n-1}\left(1+(1-p)^n\right)}$$

**A SHORTCUT: LINEARITY OF EXPECTATION**

We can skip the four cases. Useful time in the window = (useful time from long frames) + (useful time in half 1) + (useful time in half 2), and *expectations add even when the parts are dependent.*

- Long: $2T$ with probability $sz^2$, i.e. a fraction $sz^2$ of the window.
- Each half: $T$ with probability $sz$ (exactly one short node, no long node), i.e. $\frac{1}{2}sz$ of the window per half.

Total: $sz^2 + 2 \cdot \frac{1}{2}sz = sz(1+z)$. Same answer, no case analysis.

**EXAMPLE · DID SYNCHRONIZING HELP? $n = 5,\ p = 0.1$**

**After** (2c): $s = 5 \cdot 0.1 \cdot 0.9^4 = 0.328$, $z = 0.9^5 = 0.590$, so $\eta = 0.328 \cdot 0.590 \cdot 1.590 = 0.308$.
**Before** (2a, 2b): per $2T$ window, the $n$ long nodes deliver on average $n \cdot P_{\text{suc}}^{\text{slotted}} \cdot 2T$ and the $n$ short nodes, each with 2 slots, $n \cdot 2 \cdot P_{\text{suc}}^{\text{pure}} \cdot T$. Dividing by $2T$: $\eta = n\,P_{\text{suc}}^{\text{slotted}} + n\,P_{\text{suc}}^{\text{pure}} = 5 \cdot 0.0135 + 5 \cdot 0.0202 = 0.169$.
Aligning the short nodes' slots nearly doubles the goodput, the same factor of 2 as between pure and slotted ALOHA. Optimizing $p$ for $n = 100$ (numerically): about 0.20 before and 0.30 after. Even after, the mixed network stays below $1/e$: a long frame needs *both* halves free of short frames.

WEEK 02

# Practice Problems and Summary

SECTION 30

## Practice problems with full solutions

EXAMPLE · P1: LAYERS

A router receives an Ethernet frame and forwards the packet on a SONET link. Which headers does it read, and which does it replace?

**Solution.** It removes the Ethernet (link) header, reads the IP (network) header to choose the outgoing link, decrements the TTL, and wraps the datagram in a new SONET link header. It never reads the TCP or HTTP headers: transport and application exist only at the hosts.

EXAMPLE · P2: ENCAPSULATION OVERHEAD

An application sends 100 bytes over TCP/IP/Ethernet. What fraction of the frame (headers only) is data? And for 1000 bytes?

**Solution.** $100/154 = 64.9\%$. $1000/1054 = 94.9\%$.

EXAMPLE · P3: SLOTTED ALOHA

$N = 4$ nodes, each transmits in a slot with probability $p = 0.2$. (a) Probability that node 1 succeeds in a slot? (b) Goodput? (c) The best $p$ and its goodput?

**Solution.** (a) $0.2 \cdot 0.8^3 = 0.1024$. (b) $4 \times 0.1024 = 0.4096$. (c) $p^* = 1/4$: $(3/4)^3 = 0.4219$.

EXAMPLE · P4: EMPTY SLOTS

In P3 with $p = 0.2$, what fraction of slots is empty, and what fraction has collisions?

**Solution.** Empty: $0.8^4 = 0.4096$. Collisions: $1 - 0.4096 - 0.4096 = 0.1808$.

EXAMPLE · P5: PURE ALOHA

$N = 3$ nodes, $p = 0.2$. Goodput of pure ALOHA? Compare with slotted ALOHA.

**Solution.** Pure: $3 \cdot 0.2 \cdot 0.8^4 = 0.2458$. Slotted: $3 \cdot 0.2 \cdot 0.8^2 = 0.384$. Pure is lower because the vulnerable window is twice as long.

EXAMPLE · P6: CSMA/CD MINIMUM FRAME

A CSMA/CD LAN runs at 100 Mbps over at most 200 m of cable ($s = 2 \times 10^8$ m/s). What is the minimum frame size?

**Solution.** PROP $= 200/(2 \times 10^8) = 1\,\mu$s. Need TRANSP $\geq 2\,$PROP $= 2\,\mu$s, so $L \geq 2 \times 10^{-6} \times 10^8 = 200$ bits $= 25$ bytes.

EXAMPLE · P7: CSMA/CD EFFICIENCY

PROP $= 5\,\mu$s and TRANSP $= 100\,\mu$s. Find $a$ and the efficiency in both models.

**Solution.** $a = 0.05$. Simple model: $1/(1+0.15) = 87\%$. Simulation model: $1/(1+0.25) = 80\%$.

EXAMPLE · P8: WASTED SLOTS

In the CSMA/CD model, a slot is won with probability $\alpha = 0.25$. Expected wasted slots per packet?

**Solution.** $A = (1-\alpha)/\alpha = 0.75/0.25 = 3$ slots, i.e. $3 \times 2\,$PROP $= 6\,$PROP of overhead per packet.

EXAMPLE · P9: EXPONENTIAL BACKOFF

Two Ethernet NICs collide for the 3rd time on a 10 Mbps LAN. (a) From which set is $K$ chosen? (b) Probability that they collide again? (c) Longest possible wait?

**Solution.** (a) $\{0, \ldots, 7\}$. (b) They pick the same $K$ with probability $8 \cdot (1/8)^2 = 1/8$. (c) $7 \times 512 \times 0.1\,\mu$s $= 358.4\,\mu$s.

EXAMPLE · P10: HIDDEN TERMINAL

A, B and C are in a line; A and C are each 80 m from B, and the radio range is 100 m. Can A and C hear each other? Which mechanism stops their data frames from colliding at B?

**Solution.** A and C are 160 m apart, beyond the range: they are hidden from each other, so carrier sensing fails. RTS/CTS: B's CTS reaches both, so the station that did not win the CTS defers.

## Recitation problems

EXAMPLE · P11: GOODPUT IN BITS PER SECOND

A slotted ALOHA channel has $B = 1$ Mbps and frames of $L = 1000$ bits. There are 4 nodes with $p = 0.25$. (a) Slot length? (b) Goodput? (c) Useful bit rate? (d) Average time between successful frames?

**Solution.** (a) $T = L/B = 1$ ms. (b) $\eta = 4 \cdot 0.25 \cdot 0.75^3 = 0.422$. (c) $0.422 \times 1$ Mbps $= 422$ kbps. (d) View 2: $T/\eta = 1/0.422 = 2.37$ ms.

EXAMPLE · P12: WHICH LAYER?

Place in the OSI model: TLS encryption, TCP acknowledgements, Ethernet's binary exponential backoff, choosing a route between two cities.

**Solution.** TLS: presentation (6) in OSI; in the Internet stack the application provides it. TCP ACKs: transport (4). Backoff: data link (2), MAC sub-layer. Routing: network (3).

EXAMPLE · P13: POISSON LOAD

A slotted ALOHA channel carries on average $G = 0.6$ attempts per slot. (a) Fraction of slots that are successful, empty and collisions? (b) Same load on pure ALOHA?

**Solution.** (a) Success $0.6e^{-0.6} = 0.329$; empty $e^{-0.6} = 0.549$; collision $1 - 0.329 - 0.549 = 0.122$. (b) $0.6e^{-1.2} = 0.181$, almost exactly the pure maximum $1/(2e) = 0.184$, because $G = 0.6$ is close to $G^* = 0.5$.

EXAMPLE · P14: QUESTION 1 VARIANT

In Question 1, let the lower network also run *slotted* ALOHA. With $p = 0.5$, $q = 0.2$, what is C's one-shot success probability now?

**Solution.** Lower: $q(1-q)^3 = 0.2 \cdot 0.512 = 0.1024$. Total: $0.5 \cdot 0.128 + 0.5 \cdot 0.1024 = 0.115$, up from 0.090: synchronizing the lower network halves each X node's vulnerable window.

EXAMPLE · P15: VULNERABLE WINDOWS OF MIXED LENGTHS

A pure ALOHA channel carries frames of length $T$ and of length $3T$. How long is the vulnerable window of a $3T$ frame against $T$ frames, and of a $T$ frame against $3T$ frames?

**Solution.** (Own length) + (other's length) in both cases: $3T + T = 4T$. The window is the same, but a long frame occupies more of the channel each time it collides.

EXAMPLE · P16: QUESTION 2C WITH NUMBERS

In Question 2c, take $n = 10$, $p = 0.05$. Find $\eta$.

**Solution.** $s = 10 \cdot 0.05 \cdot 0.95^9 = 0.315$, $z = 0.95^{10} = 0.599$. $\eta = sz(1+z) = 0.315 \cdot 0.599 \cdot 1.599 = 0.302$.

SECTION 31

# Cheat sheet

WEEK 02 ON ONE PAGE

- Modularity: interfaces hide implementation; extensible; can cost performance. Network design choices: layering, where to implement, where to keep state.
- Layers: Physical (bits) / Link (frames, local addresses, MAC) / Network (packets, global addresses, routing, IP only) / Transport (processes; TCP, UDP) / Application. Routers: bottom 3. Hosts: all 5.
- Protocol = syntax (format, order) + semantics (meaning, actions). Standards: IETF RFCs. Layer = service + interface + protocol.
- Encapsulation: each layer adds a header. TCP 20 + IP 20 + Ethernet 14 = 54 bytes. Hourglass: IP is the narrow *waist*.
- MAC protocol: distributed; coordinates over the channel itself. Wish list: alone $\to R$; $M$ busy $\to R/M$; decentralized; no sync; simple.

- TDMA/FDMA: no collisions, fair, but $R/N$ even when alone; unused slots/bands idle.
- Slotted ALOHA: $\alpha = Np(1-p)^{N-1}$, $p^* = 1/N$, max $(1-\frac{1}{N})^{N-1} \to 1/e \approx 0.37$. Pure ALOHA: $Np(1-p)^{2(N-1)}$, max $\to 1/(2e) \approx 0.18$.
- CSMA: listen first; collisions still happen because of propagation delay. CSMA/CD: abort on collision; need TRANSP $\geq 2\,$PROP.
- CSMA/CD goodput: $A = (1-\alpha)/\alpha$; $\eta = 1/(1+3a)$ (simple), $\approx 1/(1+5a)$ (precise), $a =$ PROP/TRANSP.
- Ethernet: after the $m$-th collision $K \in \{0, \ldots, 2^m - 1\}$ (capped at $m = 10$), wait $K \cdot 512$ bit times; jam 48 bits; bit time $0.1\,\mu$s at 10 Mbps.
- MAC address: 48 bits, flat, portable, IEEE gives 24-bit prefixes. IP address: hierarchical, not portable.
- 802.11: no CD (weak received signal, hidden terminal, fading) $\Rightarrow$ CSMA/CA: DIFS, random backoff frozen while busy, ACK after SIFS; RTS/CTS reserves the channel.

## RECITATION 2 ON ONE PAGE

- OSI: Physical (bit) / Data link (frame) / Network (packet) / Transport (segment) / Session / Presentation / Application. Internet: layers 5–7 merged into the application. Wi-Fi and Ethernet are layers 1 *and* 2; MAC is a sub-layer of 2.
- $T = L/B$. Throughput: fraction of time anything is sent. Goodput $\eta$: fraction of time spent on *successful* sends. $\times B$ gives bits/s.
- Two views: $\eta = \mathbb{E}[T_{\text{suc}}]/T = P_{\text{suc}}$ (one slot), or $T/(T \cdot \mathbb{E}[\text{Geo}(P_{\text{suc}})]) = P_{\text{suc}}$ (wait for success).
- Binomial: slotted $np(1-p)^{n-1}$, $p^* = 1/n$, $\to 1/e$. Pure $np(1-p)^{2(n-1)}$, $p^* = 1/(2n-1)$, $\to 1/(2e)$.
- Poisson ($G$ attempts per slot): slotted $Ge^{-G}$, max $1/e$ at $G = 1$; pure $Ge^{-2G}$, max $1/(2e)$ at $G = \frac{1}{2}$. Binomial with $p = G/n$, $n \to \infty$.
- Vulnerable window (pure) = own frame length + other frame length: $2T$ for equal frames.
- Pure node vs. slotted grid of $2T$: half its frames inside one slot, half straddling two.
- Exam tricks: split by cases with total probability; "one specific node" $\to$ no $\binom{n}{1}$, "any node" $\to$ multiply by $n$; use linearity of expectation to avoid case analysis.